

INFORMATIQUE • TLED

MODULE 2 : SYSTÈMES D'INFORMATION ET BASES DE DONNÉES

LEÇON N06

IMPLÉMENTATION D'UNE BASE DE DONNÉES

DURÉE CONSEILLÉE : 10 H

DANS CETTE LEÇON

Cette leçon présente la mise en œuvre concrète d'une base de données relationnelle : création de la base et des tables, insertion, modification, suppression et interrogation des données à l'aide du langage SQL, en mode graphique (Access) ou en mode console (MySQL). Elle culmine avec la jointure entre deux tables, compétence clé de l'épreuve d'informatique au Baccalauréat.

- À la fin de cette leçon, je sais définir les notions de base de données, SGBD, SQL et requête, et citer des exemples de SGBD (Access, MySQL)
- À la fin de cette leçon, je sais écrire des requêtes SQL de définition des données : CREATE TABLE, ALTER TABLE, DROP TABLE
- À la fin de cette leçon, je sais choisir les types de données adaptés (VARCHAR, CHAR, INT, DATE) et poser les contraintes PRIMARY KEY, REFERENCES, NOT NULL
- À la fin de cette leçon, je sais insérer des données avec INSERT INTO
- À la fin de cette leçon, je sais modifier et supprimer des données avec UPDATE et DELETE en maîtrisant le rôle de la clause WHERE
- À la fin de cette leçon, je sais interroger une table avec SELECT en utilisant les opérateurs =, <>, <=, >=, AND, OR et les paramètres DISTINCT et *



TÉLÉCHARGE ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

1. Bases de données, SGBD et langage SQL
2. Types de données et contraintes d'intégrité
3. Requêtes de définition des données : CREATE, ALTER, DROP
4. Insertion, modification et suppression des données
5. Interrogation d'une table : SELECT
6. Jointure entre deux tables et mise en œuvre pratique
7. Activité d'intégration
8. Méthodes et exercices résolus
9. Exercices
10. Corrigés des exercices
11. Fiche bilan

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais définir les notions de base de données, SGBD, SQL et requête, et citer des exemples de SGBD (Access, MySQL)
- À la fin de cette leçon, je sais écrire des requêtes SQL de définition des données : CREATE TABLE, ALTER TABLE, DROP TABLE
- À la fin de cette leçon, je sais choisir les types de données adaptés (VARCHAR, CHAR, INT, DATE) et poser les contraintes PRIMARY KEY, REFERENCES, NOT NULL
- À la fin de cette leçon, je sais insérer des données avec INSERT INTO
- À la fin de cette leçon, je sais modifier et supprimer des données avec UPDATE et DELETE en maîtrisant le rôle de la clause WHERE
- À la fin de cette leçon, je sais interroger une table avec SELECT en utilisant les opérateurs =, <>, <=, >=, AND, OR et les paramètres DISTINCT et *
- À la fin de cette leçon, je sais écrire une requête de sélection combinant deux tables par jointure avec des alias
- À la fin de cette leçon, je sais créer une base de données et une table en mode graphique sous Access ou en mode console sous MySQL

PRÉREQUIS

- Notion de table (entité), d'attribut et de clé primaire vus en classe de Première

- Modélisation d'un MCD simple (entités et associations) vue en début d'année de Terminale
- Notion de type de donnée (entier, chaîne de caractères, date) vue en algorithmique
- Manipulation de base d'un logiciel (ouverture, enregistrement, exécution d'une commande)

1 BASES DE DONNÉES, SGBD ET LANGAGE SQL

Au lycée bilingue de Yaoundé, le censeur gère encore les 2400 élèves dans de gros cahiers cartonnés : un cahier pour les inscriptions, un autre pour les notes, un troisième pour les paiements de la scolarité. À chaque conseil de classe, il passe trois jours entiers à rechercher, recopier et recouper des informations, au risque d'oublier un élève ou de mélanger deux homonymes. Excédé, le proviseur prend une décision : le lycée se dotera d'une **base de données**, et ce sont les élèves de Terminale scientifique qui auront la mission de la construire et de l'interroger.

Comment créer les tables ? Comment y insérer les 2400 élèves ? Comment corriger une erreur de saisie sans tout recopier ? Comment retrouver en une seconde tous les élèves de Tle C nés après 2006 ? Toutes ces opérations reposent sur trois piliers : la **base de données** (les informations organisées), le **SGBD** (le logiciel qui les gère) et le langage **SQL** (le langage qui permet de dialoguer avec ce logiciel). C'est toute la puissance de ce trio que cette leçon va dévoiler, étape par étape.

1.1 Notion de base de données

Intuition : à quel problème répond une base de données ?

Imagine le cahier des inscriptions du lycée. Pour retrouver un élève, on feuillette page par page ; pour corriger une date de naissance, on rature ; pour connaître les élèves de Tle C, on relit tout le cahier. Les informations sont dupliquées (le nom d'un élève figure dans le cahier des notes et dans celui des paiements), incohérentes (une date corrigée ici, oubliée là) et fragiles (un cahier mouillé, et tout est perdu). Une base de données résout exactement ces trois maux : elle centralise les données, les structure de façon rigoureuse et les conserve de manière permanente sur un support numérique, où des milliers de recherches par seconde deviennent possibles.

DÉFINITION Base de données

Une **base de données** (en abrégé BD) est un ensemble structuré et organisé de données relatives à un même domaine, stockées de façon permanente sur un support numérique, et accessibles à plusieurs utilisateurs.

Décortiquons les quatre caractéristiques de cette définition :

- **Structurée** : les données ne sont pas jetées en vrac ; elles sont rangées dans des tables (tableaux à deux dimensions), elles-mêmes décrites par des règles précises (chaque colonne a un nom et un type).
- **Organisée autour d'un même domaine** : la base du lycée ne contient que ce qui concerne la vie scolaire (élèves, classes, notes, paiements), pas les stocks d'une boutique.
- **Permanente** : les données survivent à l'extinction de l'ordinateur, car elles sont enregistrées sur le disque dur, contrairement aux variables d'un programme qui vivent en mémoire vive.
- **Partageable** : le censeur, le comptable et le proviseur peuvent consulter la même base, chacun selon ses droits.

EXEMPLE La base fil conducteur : GESTION_LYCEE

Toute cette leçon s'appuiera sur une même base de données, nommée `GESTION_LYCEE`, qui contient (entre autres) deux tables :

- la table `ELEVE` : matricule, nom, prénom, date de naissance, sexe, code de la classe ;
- la table `CLASSE` : code de la classe, libellé (par exemple « Tle C 1 »), effectif, salle.

Chaque élève appartient à une classe : on dit que les deux tables sont reliées. Ce lien sera exploité à la fin de la leçon grâce aux jointures.

1.2 Le SGBD : le logiciel qui gère la base

Intuition.

Une base de données n'est qu'un ensemble de fichiers sur un disque : elle ne fait rien toute seule. Il faut un logiciel « chef d'orchestre » qui reçoive les demandes des utilisateurs, vérifie les droits d'accès, garantit la cohérence des données et range physiquement les informations sur le disque. Ce logiciel, c'est le SGBD.

DÉFINITION SGBD

Un **SGBD** (Système de Gestion de Bases de Données) est un logiciel qui permet de créer, manipuler et administrer une base de données : il assure le stockage, la recherche, la mise à jour, la sécurité et la cohérence des données.

Les quatre **fonctions de manipulation des données** offertes par tout SGBD sont à connaître par cœur :

Fonction	Rôle	Ordre SQL
Insertion	Ajouter de nouvelles données (inscrire un élève)	INSERT INTO
Modification	Mettre à jour des données existantes (corriger une date de naissance)	UPDATE
Sélection	Interroger la base pour extraire des informations (liste des élèves de Tle C)	SELECT
Suppression	Retirer des données (radier un élève parti)	DELETE

ATTENTION SGBD ≠ base de données

L'erreur la plus fréquente au Baccalauréat est de confondre les deux notions. La base de données est l'ensemble des données organisées (les fichiers contenant les élèves, les classes...) ; le SGBD est le logiciel qui les gère (MySQL, Access...). Dire « MySQL est une base de données » est faux : MySQL est un SGBD, qui permet de créer et d'exploiter des

bases de données.

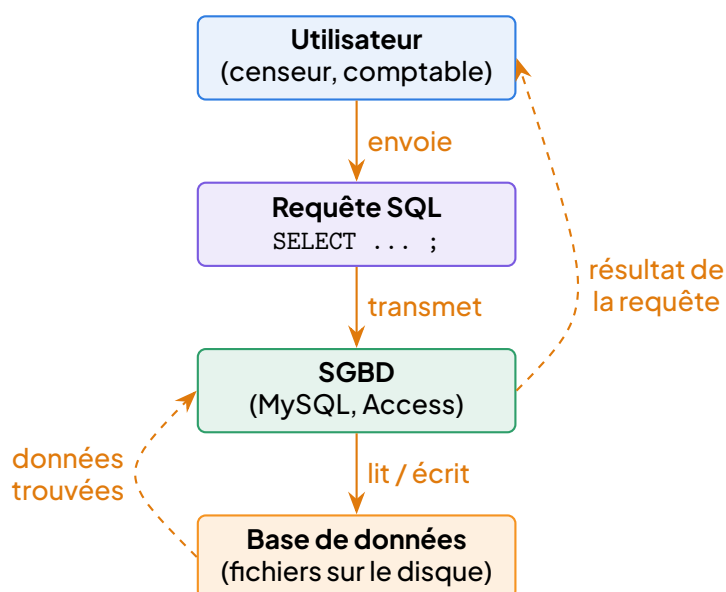
Exemples de SGBD.

Le programme officiel en retient deux, que nous utiliserons en travaux pratiques :

- **Microsoft Access** : SGBD bureautique de la suite Microsoft Office, manipulé en mode graphique (on crée les tables à la souris, on remplit des grilles). Idéal pour débiter et pour de petites bases locales (bibliothèque scolaire, boutique).
- **MySQL** : SGBD libre et gratuit, très répandu dans le monde professionnel, manipulé en mode console (on tape des instructions SQL dans une fenêtre de commandes) ou via des outils graphiques. C'est lui qui équipe la majorité des sites web et des serveurs d'entreprise.

En complément, citons **SQLite** (SGBD ultra-léger embarqué dans les téléphones Android) et **PostgreSQL** (SGBD libre très puissant, utilisé par de grandes administrations).

Critère	Microsoft Access	MySQL
Interface	Graphique (fenêtres, grilles, assistants)	Console (ligne de commandes SQL)
Mode de saisie	Clics et formulaires, peu de code	Instructions SQL tapées au clavier
Coût	Payant (licence Microsoft Office)	Gratuit et libre
Contexte d'usage	Petite base locale, un poste, bureautique	Serveur multi-utilisateurs, applications professionnelles



Architecture en couches : l'utilisateur formule une requête SQL, le SGBD l'exécute sur les fichiers de la base et renvoie le résultat.

Ce schéma est fondamental : l'utilisateur ne touche jamais directement les fichiers de la base. Il s'adresse au SGBD, dans un langage précis : le SQL.

1.3 Le langage SQL : parler au SGBD

Intuition.

Le SGBD est puissant, mais il ne comprend ni le français ni l'anglais courant. Pour lui demander « donne-moi les élèves de Tle C nés après 2006 », il faut un langage rigoureux, sans ambiguïté, avec une grammaire stricte. Ce langage existe, il est international, et il s'appelle SQL.

DÉFINITION SQL

Le langage **SQL** (Structured Query Language, « langage structuré d'interrogation ») est le langage normalisé de définition, de manipulation et d'interrogation des bases de données relationnelles.

DÉFINITION Requête

Une **requête** est une instruction SQL envoyée au SGBD pour obtenir une action (créer une table, insérer un élève, modifier une donnée) ou un résultat (la liste des élèves d'une classe).

PROPRIÉTÉ Portabilité du SQL (admise)

Tout SGBD relationnel comprend le langage SQL, qui est une norme internationale. Par conséquent, une requête écrite pour MySQL fonctionne, à quelques détails près, sous Access, PostgreSQL ou SQLite : les requêtes sont portables d'un SGBD à l'autre. Apprendre SQL, c'est apprendre à dialoguer avec tous les SGBD relationnels à la fois.

LESAVAIS-TU ? SQL, un demi-siècle de bons et loyaux services

SQL est né dans les laboratoires d'IBM au début des années 1970, sous le nom de SEQUEL, à partir des travaux du mathématicien Edgar Frank Codd sur le modèle relationnel. Normalisé dès 1986, il est devenu un standard international (ISO) régulièrement enrichi. Aujourd'hui, il est partout : les banques l'utilisent pour gérer les comptes, la CAMTEL pour gérer ses abonnés à la fibre optique, et MTN Cameroun pour suivre des millions de clients et leurs forfaits. Chaque rechargement de crédit déclenche des requêtes SQL !

Les deux grandes familles de requêtes.

Il est essentiel de distinguer, dès maintenant, deux catégories d'instructions SQL :

- Les **requêtes de définition des données** (LDD) agissent sur la structure de la base : créer une table (`CREATE TABLE`), modifier sa structure (`ALTER TABLE`), la détruire (`DROP TABLE`). Elles fabriquent les « tiroirs ».
- Les **requêtes de manipulation des données** (LMD) agissent sur le contenu des tables : insérer (`INSERT INTO`), modifier (`UPDATE`), supprimer (`DELETE`) et sélectionner (`SELECT`) des données. Elles remplissent et vident les tiroirs.

À RETENIR Convention d'écriture du SQL

- Les mots-clés du langage s'écrivent en **MAJUSCULES** : `SELECT`, `FROM`, `WHERE`, `INSERT INTO`...
- Les noms des tables et des colonnes s'écrivent en minuscules ou en majuscules, selon le choix du concepteur (nous écrirons les tables en majuscules : `ELEVE`, `CLASSE`).
- Chaque instruction se termine par un point-virgule ; : c'est lui qui signale au SGBD la fin de l'ordre.

ATTENTION Le point-virgule oublié

En mode console MySQL, si tu valides une instruction sans le ; final, le SGBD attend la suite et affiche une flèche -> : il ne s'agit pas d'une erreur, tape simplement ; puis Entrée. Autre piège : SQL n'est pas sensible à la casse pour les mots-clés (`select` fonctionne), mais respecter la convention des majuscules rend tes requêtes lisibles — et c'est ce qu'attend le correcteur au Baccalauréat.

1.4 Premier contact avec SQL

Pour fixer les idées, voici un premier aperçu des quatre fonctions de manipulation appliquées à la table `ELEVE` de notre base `GESTION_LYCEE`. Chaque instruction sera étudiée en détail dans les sections suivantes ; observe pour l'instant la logique générale : un verbe d'action, une table, des précisions.

```
-- Insertion : inscrire un nouvel eleve dans la table ELEVE
INSERT INTO ELEVE (matricule, nom, prenom, date_naiss, sexe, code_classe)
VALUES ('LB2401', 'NGONO', 'Ariane', '2006-03-15', 'F', 'TC1');

-- Selection : retrouver les eleves de Tle C1 nes apres 2006
SELECT nom, prenom
FROM ELEVE
WHERE code_classe = 'TC1' AND date_naiss > '2006-12-31';

-- Modification : corriger la date de naissance d'un eleve
UPDATE ELEVE
SET date_naiss = '2006-03-25'
WHERE matricule = 'LB2401';

-- Suppression : radier un eleve qui a quitte le lycee
```

```
DELETE FROM ELEVE  
WHERE matricule = 'LB2401';
```

Remarque la structure commune : un **mot-clé d'action** en majuscules ouvre l'instruction, le nom de la **table** suit, puis viennent les **conditions** ou les **valeurs**, et le tout se termine par un ;.

EXERCICE RÉSOLU 1 Identifier les notions dans un scénario

Le comptable du lycée ouvre le logiciel MySQL, se connecte à la base GESTION_LYCEE et tape une instruction pour obtenir la liste des élèves n'ayant pas encore payé la scolarité. Pour chaque élément souligné de cette phrase, dire s'il s'agit de la base de données, du SGBD, du langage SQL ou d'une requête.

SOLUTION

Analysons chaque élément :

1. MySQL : c'est le SGBD, c'est-à-dire le logiciel qui gère la base ;
2. GESTION_LYCEE : c'est la base de données, l'ensemble structuré des données du lycée ;
3. l'instruction tapée (par exemple `SELECT nom FROM ELEVE WHERE scolarite = 'non paye' ;`) : c'est une requête, écrite en SQL, le langage normalisé compris par le SGBD.

Ce scénario illustre la chaîne complète : l'utilisateur formule une requête en SQL, que le SGBD exécute sur la base de données pour renvoyer le résultat.

EXERCICE 1 À toi de jouer

1. Définis en une phrase chacun des termes suivants : base de données, SGBD, requête, SQL.
2. Cite les quatre fonctions de manipulation des données et l'ordre SQL correspondant à chacune.
3. Le secrétariat du lycée veut un outil simple, graphique, sur un seul poste ; le prestataire informatique de la CAMTEL veut un serveur multi-utilisateurs. Quel SGBD conseilles-tu dans chaque cas ? Justifie.
4. Classe les instructions suivantes en « définition des données » ou « manipulation des données » : `CREATE TABLE`, `SELECT`, `DROP TABLE`, `UPDATE`, `ALTER TABLE`, `INSERT INTO`.

CORRIGÉ Exercice 1

1. Base de données : ensemble structuré et organisé de données relatives à un même domaine, stockées de façon permanente. SGBD : logiciel permettant de créer, manipuler et administrer une base de données. Requête : instruction SQL envoyée au SGBD pour obtenir une action ou un résultat. SQL : langage normalisé de définition, de manipulation et d'interrogation des bases de données relationnelles.
2. Insertion (INSERT INTO), modification (UPDATE), sélection (SELECT), suppression (DELETE).
3. Secrétariat : Microsoft Access, car son interface graphique convient à un usage bureautique sur un poste unique. CAMTEL : MySQL, car c'est un SGBD serveur, gratuit, conçu pour de nombreux utilisateurs simultanés.
4. Définition des données (structure) : CREATE TABLE, DROP TABLE, ALTER TABLE. Manipulation des données (contenu) : SELECT, UPDATE, INSERT INTO.

À RETENIR L'essentiel de la section

- Une base de données est un ensemble structuré et organisé de données d'un même domaine, stockées de façon permanente.
- Un SGBD (Access, MySQL) est le logiciel qui crée, manipule et administre la base ; il offre quatre fonctions : insertion, modification, sélection, suppression.
- Le SQL est le langage normalisé pour dialoguer avec tout SGBD relationnel ; une requête est une instruction SQL.
- On distingue les requêtes de définition (structure : CREATE, ALTER, DROP) et les requêtes de manipulation (contenu : INSERT, UPDATE, DELETE, SELECT).
- Convention : mots-clés en MAJUSCULES, instruction terminée par ;.

Maintenant que le décor est planté — la base, le logiciel, le langage —, la prochaine étape consiste à apprendre à construire les tables : types de données et contraintes d'intégrité nous attendent.

2 TYPES DE DONNÉES ET CONTRAINTES D'INTÉGRITÉ

Dans la section précédente, nous avons découvert ce qu'est une **base de données**, le rôle du **SGBD** et le langage **SQL** qui permet de dialoguer avec lui. Nous savons donc désormais à qui nous adresser pour stocker les informations du lycée. Reste une question essentielle : comment décrire précisément chaque information à enregistrer ?

Imagine la secrétaire du lycée bilingue de Bafoussam qui reçoit une fiche d'inscription vierge.

Sur cette fiche, chaque case a une nature bien précise : la case « matricule » attend un code court, la case « nom » attend du texte, la case « date de naissance » attend une date, la case « effectif de la classe » attend un nombre entier. Si un élève écrit « bonjour » dans la case date de naissance, la secrétaire refuse la fiche. Le SGBD fonctionne exactement pareil : avant de créer une table, on doit déclarer le **type** de chaque colonne (ce qu'elle peut contenir) et ses **contraintes** (les règles qu'elle doit respecter). C'est l'objet de cette section.

2.1 Notion de type de donnée

DÉFINITION Type de donnée

Un **type de donnée** est la nature des valeurs qu'une colonne (attribut) d'une table est autorisée à contenir. Il est déclaré à la création de la table, juste après le nom de la colonne. Le SGBD rejette toute valeur qui ne respecte pas le type déclaré.

Pourquoi est-ce si important ? Trois raisons concrètes :

- **La cohérence** : impossible d'enregistrer le texte 'abc' dans une colonne déclarée entière ; le SGBD bloque l'erreur dès la saisie.
- **L'économie de mémoire** : un entier occupe quelques octets, un long texte beaucoup plus ; choisir le bon type évite de gaspiller l'espace disque du serveur.
- **Des calculs corrects** : on peut additionner deux colonnes de type INT, trier des dates chronologiquement, ce qui serait impossible si tout était stocké comme du texte.

Le programme officiel retient quatre types, que l'on retrouve dans Access et MySQL. Étudions-les un par un.

Le type CHAR(n) : chaîne de longueur fixe

CHAR(n) désigne une chaîne de caractères de longueur exactement égale à n . Si la valeur saisie est plus courte, le SGBD la complète automatiquement par des espaces à droite jusqu'à atteindre n caractères. Ce type convient aux codes normalisés dont la longueur est toujours la même : un code de classe 'TC1' dans un CHAR(4) sera stocké 'TC1 ' (avec un espace), un indicatif téléphonique, un code postal, etc.

Le type VARCHAR(n) : chaîne de longueur variable

VARCHAR(n) désigne une chaîne de caractères de longueur variable, limitée à au plus n caractères. Ici, aucun espace n'est ajouté : la valeur occupe exactement la place nécessaire. C'est le type idéal pour les informations textuelles dont la longueur varie d'une ligne à l'autre : un nom d'élève (VARCHAR(30)), une adresse, une ville.

MÉTHODE Choisir entre CHAR et VARCHAR

Pour décider du type d'une colonne textuelle, pose-toi une seule question : la longueur de la valeur est-elle fixe et connue à l'avance ?

1. Si la valeur a toujours la même longueur (code, matricule normalisé, indicatif), choisis `CHAR(n)` avec n égal à cette longueur exacte.
2. Si la longueur varie d'une ligne à l'autre (nom, prénom, adresse), choisis `VARCHAR(n)` avec n légèrement supérieur à la plus longue valeur prévisible.
3. En cas de doute, choisis `VARCHAR` : il est plus souple et n'ajoute jamais d'espaces parasites.

Le type INT : nombre entier

INT (abréviation de integer, entier) désigne un nombre entier relatif, sans partie décimale. On l'utilise pour tout ce qui se compte ou se numérote : un effectif de classe, une année de naissance, un numéro d'ordre, une quantité en stock dans la boutique du lycée.

Le type DATE : les dates

DATE désigne une date calendaire. En SQL, une date s'écrit entre apostrophes et au format international 'AAAA-MM-JJ' : année sur quatre chiffres, puis le mois sur deux chiffres, puis le jour sur deux chiffres, séparés par des tirets. Par exemple, le 15 mars 2007 s'écrit '2007-03-15'.

ATTENTION Le format des dates : l'erreur classique

L'erreur la plus fréquente des débutants est d'écrire une date « à la française » : '15/03/2007'. SQL refuse ce format. La date doit être écrite au format 'AAAA-MM-JJ', entre apostrophes, avec des tirets : '2007-03-15'. Retiens l'ordre : année, mois, jour, du plus grand au plus petit. Une date mal formatée provoque soit une erreur, soit pire : une date incorrecte silencieusement enregistrée.

Type	Rôle	Exemple de valeur
CHAR(n)	Chaîne de longueur fixe n , complétée par des espaces	'TC1 ' dans un CHAR(4)
VARCHAR(n)	Chaîne de longueur variable, au plus n caractères	'Ngo Bell' dans un VARCHAR(30)
INT	Nombre entier relatif	52 (effectif), 2007 (année)
DATE	Date au format 'AAAA-MM-JJ'	'2007-03-15'

2.2 Les contraintes d'intégrité

Déclarer un type ne suffit pas. Reprenons notre fiche d'inscription : la secrétaire exige aussi que la case « nom » ne soit jamais vide, que deux élèves n'aient jamais le même matricule, et que la classe indiquée existe réellement dans l'établissement. Ces règles, en SQL, s'appellent des contraintes d'intégrité.

DÉFINITION Contrainte d'intégrité

Une **contrainte d'intégrité** est une règle déclarée sur une colonne (ou une table) que le SGBD fait respecter en permanence. Toute insertion ou modification qui violerait la contrainte est rejetée avec un message d'erreur. Les contraintes garantissent l'exactitude et la cohérence des données.

La contrainte PRIMARY KEY : l'identifiant unique

Dans un lycée de 1 500 élèves, deux élèves peuvent s'appeler « Ngo Bell Marie ». Comment distinguer leurs dossiers sans ambiguïté ? Grâce au matricule : un code attribué à un élève et à lui seul. C'est exactement le rôle de la clé primaire.

DÉFINITION Clé primaire

La **clé primaire** (PRIMARY KEY) d'une table est une colonne (ou un ensemble de colonnes) qui identifie de façon unique chaque ligne de la table. Une valeur de clé primaire doit être :

- unique : deux lignes ne peuvent jamais avoir la même valeur ;
- non nulle : la valeur est obligatoirement renseignée.

PROPRIÉTÉ Unicité de la clé primaire

Une table ne peut avoir qu'une seule clé primaire. En revanche, elle peut posséder plusieurs contraintes NOT NULL et plusieurs contraintes REFERENCES sur des colonnes différentes.

La contrainte NOT NULL : la valeur obligatoire**DÉFINITION** NOT NULL

La contrainte **NOT NULL** appliquée à une colonne interdit l'absence de valeur (la valeur NULL, c'est-à-dire « vide » ou « inconnue ») pour cette colonne. Toute ligne insérée doit obligatoirement fournir une valeur.

Exemple : le nom d'un élève doit être NOT NULL (un élève sans nom n'a pas de sens dans le fichier), alors que son numéro de téléphone peut rester vide si sa famille n'en a pas.

La contrainte REFERENCES : la clé étrangère

Venons-en à la règle la plus subtile. Le lycée gère deux tables : CLASSE (les classes existantes) et ELEVE (les élèves inscrits). Chaque élève appartient à une classe : on stocke donc dans ELEVE une colonne codeClasse. Mais comment empêcher qu'on inscrive un élève dans la classe 'ZZ9' ?

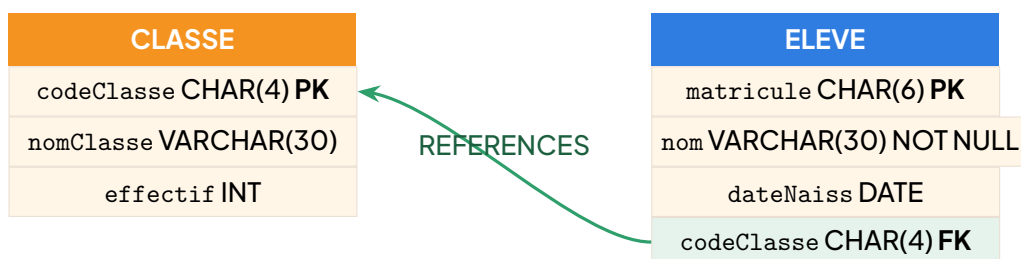
qui n'existe pas ? En déclarant que `codeClasse` de `ELEVE` référence la clé primaire de `CLASSE`.

DÉFINITION Clé étrangère

Une **clé étrangère** est une colonne d'une table qui fait référence à la clé primaire d'une autre table. Elle se déclare avec la contrainte `REFERENCES` suivie du nom de la table et de la colonne référencées :

```
codeClasse CHAR(4) REFERENCES CLASSE(codeClasse)
```

Elle garantit l'intégrité référentielle : toute valeur saisie dans la clé étrangère doit exister comme clé primaire dans la table référencée (ou être `NULL` si la colonne l'autorise).



La clé étrangère `codeClasse` de la table `ELEVE` référence la clé primaire `codeClasse` de la table `CLASSE` : tout élève est rattaché à une classe qui existe réellement.

2.3 Choix raisonné des types et des contraintes

Concevoir une table, c'est raisonner colonne par colonne. Pour chaque information, on se pose trois questions, dans l'ordre :

1. **Quelle est la nature de l'information ?** Texte de longueur fixe (`CHAR`), texte variable (`VARCHAR`), nombre entier (`INT`) ou date (`DATE`) ?
2. **Cette information identifie-t-elle la ligne ?** Si oui, c'est la clé primaire (`PRIMARY KEY`).
3. **Cette information est-elle obligatoire ?** Si oui, on ajoute `NOT NULL`. **Renvoie-t-elle vers une autre table ?** Si oui, on ajoute `REFERENCES`.

EXEMPLE La table `ELEVE` du lycée, entièrement justifiée

Appliquons la démarche à la table des élèves :

```
-- Table des classes (référéncée par ELEVE)
CREATE TABLE CLASSE (
  codeClasse CHAR(4) PRIMARY KEY, -- code fixe : 'TC1', 'TD2'
```

```
nomClasse VARCHAR(30) NOT NULL, -- libellé variable, obligatoire
effectif INT -- nombre d'élèves (peut être vide au départ)
);

-- Table des élèves
CREATE TABLE ELEVE (
    matricule CHAR(6) PRIMARY KEY, -- code fixe, unique et non nul
    nom VARCHAR(30) NOT NULL, -- longueur variable, obligatoire
    dateNaiss DATE, -- ex. '2007-03-15', peut être inconnue
    codeClasse CHAR(4) REFERENCES CLASSE(codeClasse) -- clé étrangère vers CLASSE
);
```

Chaque choix est motivé : le matricule est un code normalisé de 6 caractères (CHAR(6)) et sert d'identifiant (PRIMARY KEY); le nom varie en longueur (VARCHAR(30)) mais ne peut être vide (NOT NULL); la date de naissance est une DATE; la classe de l'élève doit exister dans CLASSE (REFERENCES).

ATTENTION Pièges fréquents sur les contraintes

- Une table ne peut avoir qu'une seule clé primaire : écrire deux fois PRIMARY KEY dans le même CREATE TABLE provoque une erreur.
- On peut en revanche multiplier les NOT NULL et les REFERENCES sur autant de colonnes que nécessaire.
- La clé étrangère et la clé primaire référencée doivent avoir des types compatibles : un CHAR(4) ne peut pas référencer un INT.
- Il faut créer la table référencée (CLASSE) avant la table qui la référence (ELEVE).

EXERCICE RÉSOLU 2 Concevoir la table LIVRE d'une bibliothèque scolaire

Énoncé : la bibliothèque du lycée veut enregistrer ses livres. Chaque livre possède un code fixe de 5 caractères (identifiant unique), un titre obligatoire d'au plus 40 caractères, une année d'édition (entier, facultative) et le code de la classe à laquelle il est affecté (code fixe de 4 caractères, devant exister dans la table CLASSE). Écris l'instruction SQL de création de la table LIVRE en justifiant chaque choix.

SOLUTION

Solution détaillée. On raisonne colonne par colonne :

- code du livre : longueur fixe de 5 caractères ⇒ CHAR(5) ; il identifie chaque livre ⇒ PRIMARY KEY.
- titre : texte de longueur variable, au plus 40 caractères ⇒ VARCHAR(40) ; obligatoire ⇒ NOT NULL.

- année d'édition : nombre entier $\Rightarrow$ INT ; facultative $\Rightarrow$ pas de NOT NULL.
- code de la classe : longueur fixe $\Rightarrow$ CHAR(4) ; doit exister dans CLASSE $\Rightarrow$ REFERENCES CLASSE(codeClasse).

```
CREATE TABLE LIVRE (  
  codeLivre CHAR(5) PRIMARY KEY,           -- identifiant unique du livre  
  titre     VARCHAR(40) NOT NULL,          -- titre obligatoire, longueur variable  
  anneeEdit INT,                           -- année d'édition (facultative)  
  codeClasse CHAR(4) REFERENCES CLASSE(codeClasse) -- classe d'affectation existante  
);
```

Vérification mentale : une seule clé primaire, les types de codeClasse sont identiques dans les deux tables (CHAR(4)), et la table CLASSE est supposée déjà créée. La requête est correcte.

À RETENIR L'essentiel de la section

- Le **type** d'une colonne définit la nature de ses valeurs : CHAR(n) (texte de longueur fixe, complété par des espaces), VARCHAR(n) (texte de longueur variable, au plus n caractères), INT (entier), DATE (date au format 'AAAA-MM-JJ').
- Les **contraintes d'intégrité** font respecter des règles : PRIMARY KEY (identifiant unique et non nul, une seule par table), NOT NULL (valeur obligatoire), REFERENCES (clé étrangère pointant vers la clé primaire d'une autre table).
- Le choix du type et des contraintes se fait colonne par colonne, selon la nature de l'information, son caractère obligatoire et ses liens avec les autres tables.

EXERCICE 2 À toi de jouer

Le proviseur veut créer la table ENSEIGNANT : matricule (code fixe de 6 caractères, identifiant), nom (obligatoire, au plus 30 caractères), date de naissance, et code de la matière enseignée (code fixe de 3 caractères référençant la clé primaire codeMat de la table MATIERE). Écris l'instruction CREATE TABLE complète, puis explique pourquoi on a choisi CHAR plutôt que VARCHAR pour le matricule.

CORRIGÉ Exercice 1

```
CREATE TABLE ENSEIGNANT (  
  matricule CHAR(6) PRIMARY KEY,  
  nom       VARCHAR(30) NOT NULL,
```

```
dateNaiss DATE,  
codeMat CHAR(3) REFERENCES MATIERE(codeMat)  
);
```

Le matricule est déclaré CHAR(6) car sa longueur est fixe et connue : tous les matricules ont exactement 6 caractères. VARCHAR serait inutile ici, puisqu'il est conçu pour les longueurs variables ; de plus, CHAR garantit qu'aucun matricule plus court ou plus long ne sera accepté.

3 REQUÊTES DE DÉFINITION DES DONNÉES : CREATE, ALTER, DROP

Dans la section précédente, tu as appris à choisir un **type de données** pour chaque attribut (CHAR, VARCHAR, INT, DATE) et à poser les **contraintes d'intégrité** qui garantissent la qualité des données (PRIMARY KEY, REFERENCES, NOT NULL). Mais tout cela restait de la théorie : nous avons dessiné des tableaux de structure sur le papier. Il est temps maintenant de fabriquer réellement les tables dans la machine. C'est le rôle des **requêtes de définition des données** (LDD, Langage de Définition de Données) : CREATE, ALTER et DROP.

Reprenons notre fil conducteur : la direction du Lycée de Nkol-Eton veut informatiser la gestion des élèves. Le MCD a été validé, le MLD a produit le schéma relationnel suivant :

```
CLASSE(codeClasse, libelle, effectifMax)  
ELEVE(matricule, nom, dateNaiss, #codeClasse)
```

Comment transformer ces deux lignes de texte en véritables tables dans MySQL ou Access ? C'est toute la question de cette section.

3.1 Création et sélection d'une base de données

Intuition. Avant de ranger des dossiers dans des tiroirs, il faut d'abord disposer d'une armoire. La **base de données** est cette armoire : c'est le conteneur qui regroupera toutes les tables du lycée (CLASSE, ELEVE, et plus tard PROFESSEUR, NOTE, etc.).

PROPRIÉTÉ Création d'une base de données

La requête de création d'une base de données s'écrit :

```
CREATE DATABASE nomBase;
```

Sous MySQL, la base créée n'est pas automatiquement active : il faut la sélectionner pour que les requêtes suivantes s'y appliquent :

```
USE nomBase;
```

EXEMPLE Création de la base du lycée

```
-- Creation de la base de donnees du Lycee de Nkol-Eton
CREATE DATABASE lycee_nkoleton;

-- Selection de la base : toutes les requetes suivantes
-- porteront sur cette base
USE lycee_nkoleton;
```

Sans le `USE`, MySQL répondrait par l'erreur « No database selected » dès que l'on tenterait de créer une table.

ATTENTION N'oublie pas le point-virgule

En mode console MySQL, chaque requête se termine par un `;` : c'est lui qui ordonne l'exécution. Si tu valides sans point-virgule, MySQL attend patiemment la suite de la requête sur la ligne suivante (le prompt affiche `->`).

3.2 Création d'une table : CREATE TABLE

Intuition. Créer une table, c'est construire un casier vide avec des colonnes étiquetées et des règles de contrôle à l'entrée. On ne met encore aucune donnée dedans : on définit uniquement la **structure** — le nom de chaque colonne (attribut), son type, et les contraintes qu'elle doit respecter.

PROPRIÉTÉ Syntaxe générale de CREATE TABLE

```
CREATE TABLE nomTable (
    attribut1 TYPE [contraintes],
    attribut2 TYPE [contraintes],
    ...
    [contraintes de table]
);
```

- `nomTable` : le nom de la table, sans espace ni accent (par convention en majuscules);
- chaque attribut est déclaré avec son **type** (`CHAR(n)`, `VARCHAR(n)`, `INT`, `DATE`, ...);
- les contraintes possibles sont `PRIMARY KEY` (clé primaire), `NOT NULL` (valeur obligatoire), `REFERENCES table(attribut)` (clé étrangère);
- les déclarations d'attributs sont séparées par des virgules; la dernière ligne avant la parenthèse fermante ne porte pas de virgule.

MÉTHODE Écrire un CREATE TABLE en quatre étapes

Face à un tableau de structure (ou à un schéma relationnel), procède toujours dans cet ordre :

1. Lister les attributs de la table, en recopiant fidèlement leurs noms depuis le MLD ;
2. Choisir le type de chaque attribut : `CHAR(n)` pour un code de longueur fixe (matricule), `VARCHAR(n)` pour un texte de longueur variable (nom), `INT` pour un nombre entier (effectif), `DATE` pour une date ;
3. Poser la clé primaire : ajouter `PRIMARY KEY` sur l'attribut souligné dans le schéma relationnel ;
4. Poser les contraintes restantes : `NOT NULL` sur les attributs obligatoires, et `REFERENCES` sur chaque clé étrangère (attribut précédé de # dans le MLD).

EXEMPLE Création des tables CLASSE et ELEVE

```
-- Etape 1 : la table referencee d'abord
CREATE TABLE CLASSE (
  codeClasse CHAR(4) PRIMARY KEY, -- ex : 'TLEC'
  libelle VARCHAR(30) NOT NULL, -- ex : 'Terminale C'
  effectifMax INT -- ex : 60
);

-- Etape 2 : la table qui reference CLASSE
CREATE TABLE ELEVE (
  matricule CHAR(6) PRIMARY KEY, -- ex : '24A001'
  nom VARCHAR(30) NOT NULL,
  dateNaiss DATE, -- format 'AAAA-MM-JJ'
  codeClasse CHAR(4) REFERENCES CLASSE(codeClasse)
);
```

La contrainte `REFERENCES CLASSE(codeClasse)` signifie : « la valeur de `codeClasse` dans `ELEVE` doit exister comme valeur de `codeClasse` dans `CLASSE` ». Impossible donc d'inscrire un élève dans une classe qui n'existe pas : c'est l'**intégrité référentielle**.

ATTENTION Crée toujours la table référencée en premier

Erreur classique du débutant : écrire `CREATE TABLE ELEVE` avant `CREATE TABLE CLASSE`. La contrainte `REFERENCES CLASSE(codeClasse)` échoue alors, car la table référencée n'existe pas encore au moment de la création. Règle d'or : on crée d'abord la table qui contient la clé primaire référencée (`CLASSE`), ensuite la table qui porte la clé étrangère (`ELEVE`).

```
-- =====
-- Script complet de creation de la base
-- Lycee de Nkol-Eton (MySQL)
-- =====
CREATE DATABASE lycee_nkoleton; -- l'armoire
USE lycee_nkoleton;           -- on ouvre l'armoire

-- Table referencee : creee EN PREMIER
CREATE TABLE CLASSE (
  codeClasse CHAR(4) PRIMARY KEY,
  libelle    VARCHAR(30) NOT NULL,
  effectifMax INT
);

-- Table referencante : creee ENSUITE
CREATE TABLE ELEVE (
  matricule CHAR(6) PRIMARY KEY,
  nom       VARCHAR(30) NOT NULL,
  dateNaiss DATE,
  codeClasse CHAR(4) REFERENCES CLASSE(codeClasse)
);
```

Script SQL commenté de création de la base et des tables CLASSE et ELEVE : l'ordre des CREATE TABLE respecte le sens des références.

3.3 Modification de la structure : ALTER TABLE

Intuition. Quelques semaines après la mise en service, la direction s'aperçoit qu'elle a besoin du sexe des élèves pour les statistiques MINEDUB, que le nom de 30 caractères est trop court pour certains élèves, et que la colonne `effectifMax` n'est finalement jamais utilisée. Faut-il tout détruire et recommencer ? Non : la requête ALTER TABLE permet de modifier la structure d'une table existante sans la supprimer.

PROPRIÉTÉ Les trois variantes de ALTER TABLE

- Ajouter une colonne : `ALTER TABLE nomTable ADD colonne TYPE ;`
- Modifier le type d'une colonne : `ALTER TABLE nomTable MODIFY colonne NOUVEAU_TYPE ;`
- Supprimer une colonne : `ALTER TABLE nomTable DROP COLUMN colonne ;`

EXEMPLE Faire évoluer la table ELEVE

```
-- Ajouter la colonne sexe ('M' ou 'F')
ALTER TABLE ELEVE ADD sexe CHAR(1);
```

```
-- Elargir le nom de 30 à 50 caractères  
ALTER TABLE ELEVE MODIFY nom VARCHAR(50) NOT NULL ;  
  
-- Supprimer la colonne effectifMax devenue inutile  
ALTER TABLE CLASSE DROP COLUMN effectifMax ;
```

ATTENTION Réduire un type peut détruire des données

Si tu modifies `VARCHAR(50)` en `VARCHAR(10)` alors que certains noms stockés dépassent 10 caractères, ils seront tronqués. Avant un `MODIFY` qui réduit une taille, vérifie toujours le contenu existant.

3.4 Suppression d'une table : DROP TABLE

Intuition. `DROP TABLE` est le bulldozer du SQL : il rase la table entière — la structure et toutes les données qu'elle contient. Il n'y a pas de corbeille, pas de message « êtes-vous sûr ? » en mode console : la suppression est immédiate et définitive.

PROPRIÉTÉ Suppression d'une table

```
DROP TABLE nomTable ;
```

Cette requête supprime définitivement la table : sa structure, ses données et ses contraintes disparaissent ensemble.

Ordre de suppression. C'est l'inverse de l'ordre de création. Si tu tentes de supprimer `CLASSE` alors que `ELEVE` contient encore une clé étrangère qui la référence, MySQL refuse (pour protéger l'intégrité référentielle). Il faut donc :

1. supprimer d'abord la table qui contient la clé étrangère : `DROP TABLE ELEVE ;`
2. supprimer ensuite la table référencée : `DROP TABLE CLASSE ;`

ATTENTION DROP TABLE n'est pas DELETE

Ne confonds jamais ces deux requêtes :

- `DELETE FROM ELEVE ;` vide la table de ses lignes, mais la table (sa structure) continue d'exister — on pourra y insérer de nouveaux élèves ;
- `DROP TABLE ELEVE ;` supprime la table elle-même : structure et données. Après cette requête, la table n'existe plus du tout.

Et rappelle-toi : `DROP TABLE` est irréversible. En entreprise, on fait une sauvegarde avant toute suppression.

EXERCICE RÉSOLU 3 Écrire une requête de création à partir d'un tableau de structure

Énoncé : le tableau ci-dessous décrit la structure de la table PROFESSEUR du lycée. Écris la requête SQL de création de cette table.

Champ	Type	Remarque
idProf	CHAR(4)	clé primaire
nomProf	VARCHAR(30)	obligatoire
dateEmbauche	DATE	facultative

SOLUTION

Solution détaillée : on applique la méthode en quatre étapes.

1. Attributs : idProf, nomProf, dateEmbauche.
2. Types : déjà donnés par le tableau — CHAR(4), VARCHAR(30), DATE.
3. Clé primaire : idProf reçoit PRIMARY KEY.
4. Contraintes : « obligatoire » se traduit par NOT NULL sur nomProf ; il n'y a pas de clé étrangère ici, donc pas de REFERENCES.

```
CREATE TABLE PROFESSEUR (  
    idProf      CHAR(4) PRIMARY KEY,  
    nomProf     VARCHAR(30) NOT NULL,  
    dateEmbauche DATE  
);
```

Vérification : trois attributs, deux virgules (jamais après le dernier), point-virgule final. La requête est correcte.

3.5 Création d'une table en mode graphique sous Access

Sous Microsoft Access, point de console : la création se fait en mode graphique, ce qui est plus intuitif pour un débutant. Voici la démarche complète pour créer la table ELEVE.

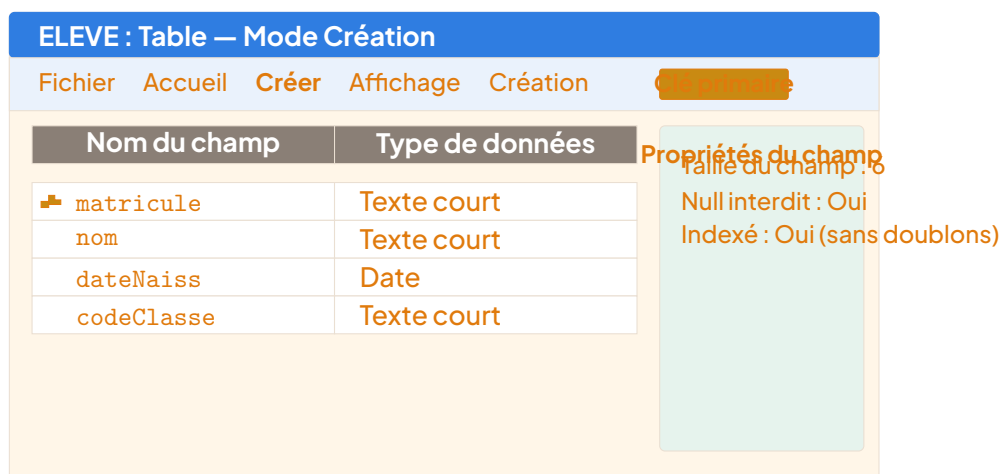
EXPÉRIENCE Créer la table ELEVE sous Access

Objectif : créer la table ELEVE sans écrire une seule ligne de SQL.
Logiciel : Microsoft Access (version 2016 ou ultérieure).

1. Lance Access, choisis Base de données vide, nomme le fichier `lycee.accdb` et clique sur Créer.
2. Dans le ruban, onglet Créer, clique sur Création de table : une grille à deux colonnes apparaît (Nom du champ / Type de données).
3. Saisis les champs un par un : `matricule` de type Texte court (dans le volet Propriétés du champ, règle Taille du champ à 6) ; `nom` de type Texte court (taille 30, Null interdit : Oui) ; `dateNaiss` de type Date/Heure ; `codeClasse` de type Texte court (taille 4).
4. Clique sur la ligne `matricule`, puis sur l'icône Clé primaire (petite clé jaune) du ruban : une clé apparaît devant le nom du champ.
5. Enregistre la table (Ctrl+S) sous le nom `ELEVE`.

Observations : Access traduit tes choix graphiques en SQL : Texte court correspond à VARCHAR, Numérique à INT, Date/Heure à DATE.

Interprétation : le mode graphique n'est qu'une interface ; en arrière-plan, le SGBD exécute exactement un `CREATE TABLE`.



Fenêtre Access en mode Création de table : saisie des noms de champs, choix des types, et clé primaire (icône en forme de clé) posée sur `matricule`. Le volet Propriétés montre Null interdit : Oui pour le champ `nom` (contrainte NOT NULL).

LESAVAIS-TU ? Une question classique du Bac

Au Baccalauréat (séries C, D et E), une question récurrente de l'épreuve d'informatique donne un tableau de structure (nom des champs, types, remarques) et demande : « Écris la requête SQL permettant de créer cette table. » Les points portent sur : le mot-clé `CREATE TABLE`, les types corrects, la clause `PRIMARY KEY`, et le `NOT NULL` quand le tableau précise «

obligatoire ». Entraîne-toi avec la méthode en quatre étapes : c'est une question où l'on ne doit perdre aucun point !

À RETENIR L'essentiel de la section

- `CREATE DATABASE nomBase ;` crée la base ; `USE nomBase ;` la sélectionne sous MySQL.
- `CREATE TABLE` définit la structure : attributs, types (`CHAR`, `VARCHAR`, `INT`, `DATE`) et contraintes (`PRIMARY KEY`, `NOT NULL`, `REFERENCES`).
- On crée d'abord la table référencée (CLASSE), ensuite celle qui porte la clé étrangère (ELEVE).
- `ALTER TABLE ... ADD / MODIFY / DROP COLUMN` fait évoluer la structure sans détruire la table.
- `DROP TABLE` supprime définitivement structure et données ; on supprime d'abord la table contenant la clé étrangère. À ne pas confondre avec `DELETE`, qui ne supprime que des lignes.
- Sous Access, le mode Création permet de construire la table graphiquement : champs, types (Texte court, Numérique, Date/Heure) et clé primaire.

EXERCICE 3 À toi de jouer

La bibliothèque du Lycée de Nkol-Eton veut gérer ses ouvrages. Le MLD donne : `LIVRE(codeLivre, titre, auteur, anneeParution)` avec `codeLivre` de type `CHAR(5)`, `titre` et `auteur` de type `VARCHAR(40)` (`titre` obligatoire), `anneeParution` entier.

1. Écris la requête de création de la table `LIVRE`.
2. La bibliothécaire souhaite ajouter un champ `editeur` de type `VARCHAR(30)`. Écris la requête.
3. Finalement, le champ `anneeParution` est jugé inutile. Écris la requête pour le supprimer.

CORRIGÉ Exercice : table `LIVRE`

1. Création (méthode en quatre étapes : attributs, types, clé primaire, `NOT NULL`) :

```
CREATE TABLE LIVRE (  
    codeLivre CHAR(5) PRIMARY KEY,  
    titre VARCHAR(40) NOT NULL,
```

```
    auteur    VARCHAR(40),  
    anneeParution INT  
);
```

2. Ajout du champ éditeur :

```
ALTER TABLE LIVRE ADD editeur VARCHAR(30);
```

3. Suppression du champ anneeParution :

```
ALTER TABLE LIVRE DROP COLUMN anneeParution;
```

4 INSERTION, MODIFICATION ET SUPPRESSION DES DONNÉES

Après avoir défini la structure de nos tables avec les commandes `CREATE TABLE`, `ALTER TABLE` et `DROP TABLE` (section précédente), notre base de données est prête, mais elle est vide. Une base de données sans données est comme une bibliothèque sans livres : l'armature existe, mais l'utilité est nulle. Cette section aborde le cœur du métier : **peupler les tables, corriger les erreurs et nettoyer les informations obsolètes**. Nous allons manipuler le langage **DML** (Data Manipulation Language) avec les trois ordres fondamentaux : `INSERT`, `UPDATE` et `DELETE`.

4.1 Insertion de données : la commande `INSERT INTO`

Intuition et besoin concret.

Imaginez la secrétaire du Lycée Bilingue de Nkolbisson qui enregistre un nouvel élève, NGONO Aline, en Terminale C1. Elle remplit une fiche papier : matricule, nom, date de naissance, classe. En SQL, cette action correspond à l'ajout d'une **ligne** (un tuple) dans la table `ELEVE`. La commande `INSERT INTO` est l'équivalent numérique de ce geste.

DÉFINITION Commande `INSERT INTO`

L'instruction `INSERT INTO` permet d'ajouter une ou plusieurs nouvelles lignes dans une table existante. Elle existe sous deux formes principales :

1. Insertion explicite (recommandée) : on précise la liste des colonnes ciblées. Cela protège contre les erreurs si la structure de la table évolue (ajout d'une colonne avec valeur par défaut).
2. Insertion implicite : on omet la liste des colonnes ; il faut alors fournir une valeur pour chaque colonne de la table, dans l'ordre exact de leur définition (`CREATE TABLE`).

PROPRIÉTÉ Syntaxe générale de INSERT INTO

```
-- Forme explicite (PRÉFÉRÉE)
INSERT INTO nom_table (colonne1, colonne2, colonne3, ...)
VALUES (valeur1, valeur2, valeur3, ...);

-- Forme implicite (RISQUÉE si structure change)
INSERT INTO nom_table
VALUES (valeur1, valeur2, valeur3, ...);
```

À RETENIR Règles de correspondance et formatage des valeurs

- **Ordre strict** : La liste des valeurs dans VALUES doit correspondre exactement (ordre, nombre, type) à la liste des colonnes (explicite ou implicite).
- **Chaînes de caractères (VARCHAR, CHAR) et dates (DATE)** : Obligatoirement encadrées par des apostrophes simples ('...'). Exemple : 'NGONO Aline', '2007-05-12' (format ISO AAAA-MM-JJ).
- **Nombres (INT, DECIMAL, FLOAT)** : Sans apostrophes. Exemple : 15000, 12.50.
- **Valeur NULL** : Saisir le mot-clé NULL (sans guillemets) pour une colonne autorisant l'absence de valeur.

MÉTHODE Écrire un INSERT INTO sans erreur

1. Identifier la table cible (ex : ELEVE).
2. Lister les colonnes à renseigner (toutes sauf celles avec AUTO_INCREMENT ou valeur par défaut gérée par le SGBD). Écrire : INSERT INTO ELEVE (matricule, nom, dateNaiss, codeClasse).
3. Préparer les valeurs dans le même ordre : chaînes/dates entre ' ', nombres nus, NULL si besoin.
4. Vérifier les contraintes : La clé primaire (ex : matricule) est-elle unique ? La clé étrangère (ex : codeClasse) existe-t-elle dans la table CLASSE ?
5. Exécuter et vérifier le message 1 row(s) affected (MySQL) ou Query OK.

EXEMPLE Insertion d'un élève complet

Supposons la table `ELEVE` créée précédemment : `CREATE TABLE ELEVE (matricule CHAR(6) PRIMARY KEY, nom VARCHAR(50) NOT NULL, dateNaiss DATE, codeClasse CHAR(4) REFERENCES CLASSE(codeClasse));`

On insère NGONO Aline, née le 12 mai 2007, en classe TC1 (qui existe dans `CLASSE`).

```
INSERT INTO ELEVE (matricule, nom, dateNaiss, codeClasse)
VALUES ('E0012', 'NGONO Aline', '2007-05-12', 'TC1');
```

Analyse : 'E0012' (CHAR), 'NGONO Aline' (VARCHAR), '2007-05-12' (DATE) sont entre apostrophes. L'ordre respecte la liste des colonnes. Si TC1 n'existe pas dans `CLASSE`, le SGBD rejette l'insertion (erreur de clé étrangère).

ATTENTION Pièges fréquents sur INSERT

- Oublier les apostrophes sur une date ou une chaîne : `VALUES (E0012, NGONO Aline, 2007-05-12, TC1)` → Erreur de syntaxe (le SGBD croit que ce sont des noms de colonnes ou des calculs : 2007 moins 5 moins 12).
- Mettre des apostrophes sur un entier : `VALUES ('E0012', 'NGONO', '2007-05-12', '15')` pour un `INT` → Fonctionne souvent par conversion implicite, mais c'est une mauvaise pratique source de bugs (comparaison chaîne vs nombre).
- Ordre inversé : `INSERT INTO ELEVE (nom, matricule) VALUES ('E0012', 'NGONO')` → Le matricule reçoit le nom et vice-versa. Données corrompues silencieusement !
- Violation de contrainte : Clé primaire dupliquée (Duplicate entry 'E0012' for key 'PRIMARY') ou clé étrangère inexistante (Cannot add or update a child row: a foreign key constraint fails).

4.2 Modification de données : la commande UPDATE

Intuition et besoin concret.

L'élève NGONO Aline change de classe : elle passe de TC1 à TC2 (redoublement ou réorientation). On ne supprime pas sa fiche pour en recréer une (on perdrait son historique, son matricule). On **modifie** la valeur de la colonne `codeClasse` pour **sa** ligne uniquement.

DÉFINITION Commande UPDATE

L'instruction `UPDATE` modifie la valeur d'un ou plusieurs attributs pour un ensemble de lignes sélectionnées par une condition. C'est une opération irréversible (sauf transaction `ROLLBACK` non vue au programme).

PROPRIÉTÉ Syntaxe générale de UPDATE

```
UPDATE nom_table
SET colonne1 = nouvelleValeur1,
    colonne2 = nouvelleValeur2,
    ...
WHERE condition_de_selection;
```

À RETENIR La clause WHERE : garde-fou vital

- Avec WHERE : Seules les lignes satisfaisant la condition sont modifiées. C'est le cas normal (ex : WHERE matricule = 'E0012').
- Sans WHERE : TOUTES les lignes de la table sont modifiées. UPDATE ELEVE SET codeClasse = 'TC2'; → Tous les élèves du lycée passent en TC2. Catastrophe administrative garantie.

EXEMPLE Changement de classe pour un élève

```
-- 1. Vérification préalable (BONNE PRATIQUE)
SELECT matricule, nom, codeClasse FROM ELEVE WHERE matricule = 'E0012';
-- Résultat : E0012 | NGONO Aline | TC1

-- 2. Modification ciblée
UPDATE ELEVE
SET codeClasse = 'TC2'
WHERE matricule = 'E0012';

-- 3. Contrôle post-modification
SELECT matricule, nom, codeClasse FROM ELEVE WHERE matricule = 'E0012';
-- Résultat : E0012 | NGONO Aline | TC2
```

Matricule	Nom	DateNaiss	CodeClasse
E0012	NGONO Aline	2007-05-12	TC1
E0013	MBARGA Paul	2006-11-03	TC1
E0014	ESSOMBA Marie	2007-01-20	TC2

Table ELEVE avant UPDATE (ligne cible en orange)

Matricule	Nom	DateNaiss	CodeClasse
E0012	NGONO Aline	2007-05-12	TC2
E0013	MBARGA Paul	2006-11-03	TC1
E0014	ESSOMBA Marie	2007-01-20	TC2

Table ELEVE après UPDATE SET codeClasse='TC2' WHERE matricule='E0012' (ligne modifiée en vert)

ATTENTION Erreur classique : UPDATE sans WHERE

```
-- DANGEREUX !!! Ne JAMAIS faire cela sans réfléchir longuement
UPDATE ELEVE SET codeClasse = 'TC2';
-- Résultat : 1500 lignes modifiées. Tous les élèves sont en TC2.
-- Annulation impossible sans sauvegarde (BACKUP).
```

Règle d'or : Avant tout UPDATE, écrivez et testez le SELECT correspondant avec le même WHERE. Si le SELECT retourne les bonnes lignes, le UPDATE les modifiera.

4.3 Suppression de données : la commande DELETE

Intuition et besoin concret.

Un élève quitte définitivement l'établissement (déménagement, exclusion définitive). Sa fiche doit disparaître de la table ELEVE pour ne plus apparaître dans les listes de classe, les bulletins, les effectifs. C'est le rôle de DELETE.

DÉFINITION Commande DELETE

L'instruction DELETE FROM supprime définitivement un ensemble de lignes d'une table. Comme UPDATE, elle opère sur les lignes sélectionnées par une clause WHERE.

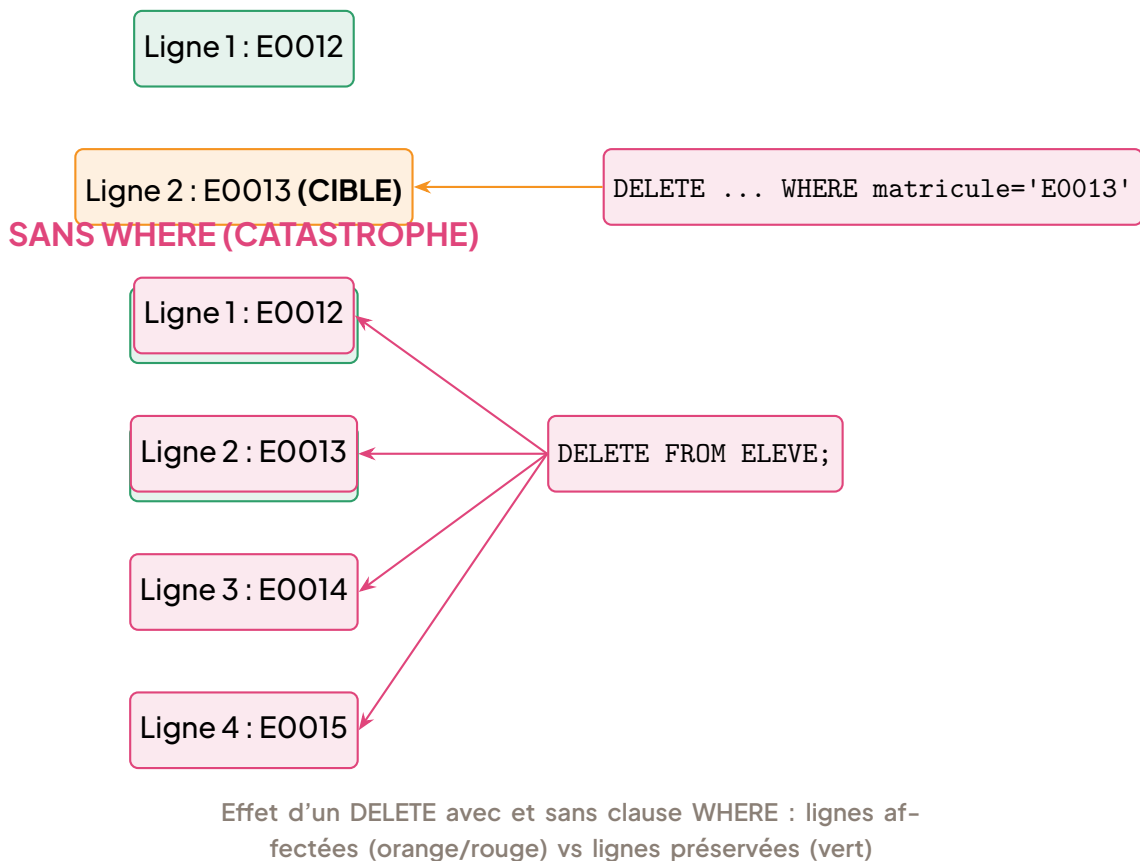
PROPRIÉTÉ Syntaxe générale de DELETE

```
DELETE FROM nom_table
WHERE condition_de_selection;
```

À RETENIR DELETE sans WHERE = Table vide

- `DELETE FROM ELEVE WHERE matricule = 'E0013';` → Supprime une ligne (celle d'Aline).
- `DELETE FROM ELEVE;` → Supprime toutes les lignes. La table existe toujours (structure intacte), mais elle est vide. Irréversible.

AVEC WHERE (SÉCURISÉ)



EXEMPLE Suppression d'un élève parti

```
-- 1. Vérification (TOUJOURS)
SELECT * FROM ELEVE WHERE matricule = 'E0013';
-- Résultat : MBARGA Paul, TC1

-- 2. Suppression
DELETE FROM ELEVE WHERE matricule = 'E0013';

-- 3. Vérification
SELECT * FROM ELEVE WHERE matricule = 'E0013';
-- Résultat : Empty set (0.00 sec)
```

ATTENTION Contraintes d'intégrité et suppression

- Clé étrangère (FK) : Si `ELEVE.codeClasse` référence `CLASSE.codeClasse`, on ne peut pas supprimer la classe TC1 tant qu'un élève y est inscrit (ERROR 1451: Cannot delete or update a parent row).
- Solutions : Supprimer d'abord les élèves de cette classe, ou modifier leur classe, ou définir `ON DELETE CASCADE` / `ON DELETE SET NULL` à la création de la FK (niveau avancé).
- Clé primaire (PK) : On ne peut pas insérer deux fois le même matricule. `INSERT` échouera sur `Duplicate entry`.

4.4 Exemple résolu complet : Gestion d'une mini-bibliothèque scolaire**EXERCICE RÉSOLU 4** Cycle de vie d'un ouvrage : Insert, Update, Delete

Contexte : La bibliothèque du Lycée de Nkongsamba gère les ouvrages. Table `OUVRAGE` :

```
CREATE TABLE OUVRAGE (code CHAR(5) PRIMARY KEY, titre VARCHAR(100) NOT NULL,
auteur VARCHAR(50), anneeEdition INT, statut VARCHAR(20) DEFAULT 'DISPONIBLE');
```

Étape 1 : Acquisition de 3 nouveaux livres (INSERT).

```
INSERT INTO OUVRAGE (code, titre, auteur, anneeEdition) VALUES
('L001', 'Algorithmique en C', 'KAMGA Jean', 2020),
('L002', 'Physique Terminale', 'FOUDA Marie', 2022),
('L003', 'Chimie Organique', 'NGUEMA Paul', 2019);
-- Note : 'statut' prend la valeur par défaut 'DISPONIBLE'.
```

Étape 2 : Le livre L002 est emprunté (UPDATE).

```
UPDATE OUVRAGE SET statut = 'EMPRUNTE' WHERE code = 'L002';
```

Étape 3 : Le livre L001 est perdu (dégât des eaux), on le retire du catalogue (DELETE).

```
DELETE FROM OUVRAGE WHERE code = 'L001';
```

Étape 4 : État final de la table.

```
SELECT * FROM OUVRAGE;
```

code	titre	auteur	anneeEdition	statut
L002	Physique Terminale	FOUDA Marie	2022	EMPRUNTE
L003	Chimie Organique	NGUEMA Paul	2019	DISPONIBLE

SOLUTION

Analyse pédagogique :

1. L'INSERT multiple (séparé par des virgules) est plus performant que 3 instructions séparées.
2. L'UPDATE cible uniquement L002 grâce à la PK. Sans WHERE, les 3 livres seraient passés en EMPRUNTE.
3. Le DELETE retire L001 définitivement. La PK L001 devient réutilisable (sauf contrainte métier).
4. Le DEFAULT sur statut évite de le répéter à l'insertion.

4.5 Mise en œuvre pratique : MySQL Console vs Access Mode Graphique

Le programme exige de savoir créer une base et une table **en mode console (MySQL)** et **en mode graphique (Access)**. Pour la manipulation de données (DML), la console MySQL reste l'outil de référence pour l'apprentissage du SQL standard.

EXPÉRIENCE TP Guidé : Manipulation DML sur MySQL Console

Objectif : Créer une base LYCEE_DB, la table ELEVE, et exécuter le cycle Insert/Update/Delete. Matériel : Poste de travail avec MySQL Server installé (XAMPP, WAMP, ou MySQL Community Server). Manipulation :

1. Ouvrir l'invite de commande (Windows : Win+R → cmd).
2. Se connecter : `mysql -u root -p` (Entrer le mot de passe, souvent vide en local).
3. Créer et sélectionner la base :

```
CREATE DATABASE LYCEE_DB CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;  
USE LYCEE_DB;
```

4. Créer la table CLASSE (référéncée par FK) puis ELEVE :

```
CREATE TABLE CLASSE (  
  codeClasse CHAR(4) PRIMARY KEY,  
  libelle VARCHAR(30) NOT NULL,  
  niveau VARCHAR(10)  
);  
  
CREATE TABLE ELEVE (  
  matricule CHAR(6) PRIMARY KEY,  
  nom VARCHAR(50) NOT NULL,  
  dateNaiss DATE,  
  codeClasse CHAR(4),  
  CONSTRAINT fk_eleve_classe FOREIGN KEY (codeClasse) REFERENCES CLASSE(codeClasse)  
);
```

5. Insérer des classes (prérequis pour la FK) :

```
INSERT INTO CLASSE VALUES ('TC1', 'Terminale C1', 'Terminale'), ('TC2', 'Terminale C2', 'Terminale');
```

6. Tester l'INSERT élève (réussi) :

```
INSERT INTO ELEVE VALUES ('E001', 'ABENA Marc', '2006-03-15', 'TC1');
```

7. Tester l'INSERT élève en erreur (FK) :

```
INSERT INTO ELEVE VALUES ('E002', 'BELA Claire', '2007-01-10', 'TD1');  
-- ERREUR 1452 (23000) : Cannot add or update a child row : a foreign key constraint fails...
```

8. Tester UPDATE avec WHERE :

```
UPDATE ELEVE SET codeClasse = 'TC2' WHERE matricule = 'E001';  
SELECT * FROM ELEVE; -- Vérifier : Marc est maintenant en TC2
```

9. Tester DELETE avec WHERE :

```
DELETE FROM ELEVE WHERE matricule = 'E001';  
SELECT * FROM ELEVE; -- Table vide
```

10. Quitter : EXIT;

Observations attendues : Les messages Query OK, 1 row affected confirment le succès. Les messages d'erreur ERROR 1452 (FK) ou ERROR 1062 (PK dupliquée) montrent l'application des contraintes d'intégrité. Interprétation : Le SGBD garantit la cohérence des données (intégrité référentielle) en bloquant les opérations invalides. La clause WHERE est le scalpel du chirurgien : sans elle, c'est la tronçonneuse.

LE SAVAIS-TU ? Le saviez-vous ? : Le coût d'un DELETE sans WHERE

Selon une étude de Verizon Data Breach Investigations Report, près de 20% des incidents de perte de données critiques en entreprise sont dus à des erreurs humaines, dont la majorité sont des UPDATE ou DELETE exécutés sans clause WHERE en environnement de production. Les administrateurs de bases de données (DBA) expérimentés appliquent la règle du "Select d'abord" : « Je ne lance jamais un UPDATE ou DELETE sans avoir copié-collé son WHERE dans un SELECT pour vérifier les lignes ciblées ». Dans les grands systèmes (banques, télécoms comme CAMTEL), on utilise des transactions (BEGIN; ... COMMIT;/ROLLBACK;) pour pouvoir annuler une erreur, mais au Bac, on apprend la rigueur avant d'exécuter !

EXERCICE 4 Entraînement au Baccalauréat

Soit la table PRODUIT (ref CHAR(5) PK, libelle VARCHAR(30), prixU DECIMAL(10,2), stock

INT, cat CHAR(3)).

1. Écrire l'instruction `INSERT` pour ajouter le produit ('P001', 'Clavier USB', 12500.00, 50, 'INF').
2. Écrire l'`UPDATE` pour augmenter de 10% le prix de tous les produits de la catégorie 'INF'.
3. Écrire le `DELETE` pour supprimer les produits dont le stock est à 0.
4. Quel serait l'effet de `UPDATE PRODUIT SET prixU = 0; (sans WHERE)?`

CORRIGÉ Exercice n°1

1. `INSERT INTO PRODUIT (ref, libelle, prixU, stock, cat) VALUES ('P001', 'Clavier USB', 12500.00, 50, 'INF');` (Nombres sans guillemets, chaînes/date avec).
2. `UPDATE PRODUIT SET prixU = prixU * 1.10 WHERE cat = 'INF';` (Calcul sur la colonne existante, condition sur catégorie).
3. `DELETE FROM PRODUIT WHERE stock = 0;` (Suppression conditionnelle).
4. Catastrophe : Le prix de tous les produits (toutes catégories confondues) devient 0 FCFA. La base de données est corrompue logiquement. Restauration depuis sauvegarde nécessaire.

5 INTERROGATION D'UNE TABLE : SELECT

Après avoir appris à créer la structure de nos tables (`CREATE TABLE`), à la modifier (`ALTER TABLE`) et à y insérer, modifier ou supprimer des lignes (`INSERT`, `UPDATE`, `DELETE`), nous arrivons à l'opération la plus fréquente et la plus puissante : **interroger la base de données pour en extraire de l'information**. C'est le cœur du métier de développeur ou d'administrateur de bases de données : transformer des données brutes stockées sur disque en réponses exploitables pour un tableau de bord, un bulletin de notes, un état de stock ou une facture.

5.1 Projection et restriction : les deux piliers du SELECT

Toute requête `SELECT` répond à deux questions fondamentales : quelles colonnes veux-je voir ? (projection) et quelles lignes m'intéressent ? (restriction). Comprendre cette distinction est la clé pour écrire des requêtes justes et efficaces.

DÉFINITION Projection (choix des colonnes)

La **projection** consiste à sélectionner un sous-ensemble des attributs (colonnes) d'une relation (table). En SQL, elle se réalise en listant les noms des colonnes souhaitées après le mot-clé `SELECT`. Le résultat est une table qui ne contient que ces colonnes, dans l'ordre indiqué.

DÉFINITION Restriction (choix des lignes)

La **restriction** consiste à ne conserver que les tuples (lignes) qui satisfont un prédicat (une condition booléenne). En SQL, elle s'exprime dans la clause `WHERE`. Le résultat est une table qui a les mêmes colonnes que la table source, mais moins de lignes (ou autant, ou zéro).

À RETENIR Ordre logique d'exécution mental

Même si on l'écrit `SELECT ... FROM ... WHERE ...`, le SGBD traite logiquement la requête dans cet ordre :

1. `FROM` : identifier la table source.
2. `WHERE` : filtrer les lignes (restriction).
3. `SELECT` : projeter les colonnes (projection).

C'est pourquoi on ne peut pas utiliser un alias de colonne défini dans le `SELECT` à l'intérieur du `WHERE` de la même requête.

5.2 Syntaxe générale et l'étoile (*)

La syntaxe canonique d'une requête de sélection sur une seule table est :

```
SELECT [DISTINCT] liste_attributs  
FROM nom_table  
[WHERE condition];
```

- `SELECT` : mot-clé obligatoire qui ouvre la requête.
- `DISTINCT` : optionnel, élimine les lignes dupliquées dans le résultat (voir plus loin).
- `liste_attributs` : noms des colonnes séparés par des virgules, ou l'étoile `*`.
- `FROM nom_table` : obligatoire, indique la table source.
- `WHERE condition` : optionnel, filtre les lignes.

PROPRIÉTÉ L'étoile * : toutes les colonnes

L'astérisque * est un raccourci qui signifie « tous les attributs de la table dans l'ordre de leur définition ».

```
SELECT * FROM ELEVE;
```

Attention : En production (application réelle), on évite `SELECT *` car :

1. Cela transfère plus de données que nécessaire (réseau, mémoire).
2. Si la structure de la table change (ajout/suppression de colonne), l'application peut planter ou afficher des données inattendues.
3. Cela rend le code moins lisible (on ne sait pas quelles colonnes sont utilisées).

Au Bac et en TP : `SELECT *` est parfaitement acceptable pour explorer une table ou pour des exercices simples.

EXEMPLE Table de référence : ELEVE

Pour tous nos exemples, nous utiliserons la table `ELEVE` suivante, peuplée de 6 lignes. Les dates suivent le format standard SQL `AAAA-MM-JJ`.

matricule	nom	prenom	sexe	dateNaiss	codeClasse
E001	MBARGA	Jean	M	2006-05-12	TC1
E002	ETOUNDI	Marie	F	2007-11-23	TC1
E003	FOPA	Pierre	M	2006-02-18	TD2
E004	KAMGA	Grace	F	2008-01-30	TC1
E005	NGONO	Paul	M	2007-09-05	TD2
E006	BELLA	Anne	F	2006-12-01	TE3

Table `ELEVE` : 6 élèves, 3 classes (TC1, TD2, TE3)

5.3 La projection explicite : choisir ses colonnes

Au lieu de *, on liste les colonnes désirées. L'ordre dans la liste `SELECT` détermine l'ordre d'affichage dans le résultat.

EXEMPLE Projection simple

Consigne : Afficher seulement le nom, le prénom et la classe de tous les élèves.

```
SELECT nom, prenom, codeClasse  
FROM ELEVE;
```

Résultat (table de projection) :

nom	prenom	codeClasse
MBARGA	Jean	TC1
ETOUNDI	Marie	TC1
FOPA	Pierre	TD2
KAMGA	Grace	TC1
NGONO	Paul	TD2
BELLA	Anne	TE3

Résultat : 6 lignes, 3 colonnes. L'ordre des colonnes suit la liste du SELECT.

5.4 La restriction avec WHERE : filtrer les lignes

La clause WHERE ne garde que les lignes pour lesquelles la condition est VRAI (TRUE). Les opérateurs de comparaison autorisés au programme sont : =, <> (différent), <, >, <=, >=.

PROPRIÉTÉ Opérateurs de comparaison en SQL

- = : égalité (fonctionne sur nombres, chaînes, dates).
- <> : différence (standard SQL ; != existe souvent mais n'est pas au programme).
- <, >, <=, >= : ordre strict ou large (numérique, alphabétique, chronologique).

Règles sur les chaînes : En SQL standard, la comparaison de chaînes peut être sensible à la casse selon le collation de la base. Au programme, on considère généralement une correspondance exacte. On écrit les littéraux chaînes entre guillemets simples : 'TC1'. Les dates s'écrivent aussi entre guillemets simples au format ISO : '2007-01-01'.

EXEMPLE Restriction simple sur une chaîne

Consigne : Lister les élèves de la classe TC1.

```
SELECT matricule, nom, prenom  
FROM ELEVE  
WHERE codeClasse = 'TC1';
```

Résultat :

matricule	nom	prenom
E001	MBARGA	Jean
E002	ETOUNDI	Marie
E004	KAMGA	Grace

3 lignes sur 6 satisfont la condition.

EXEMPLE Restriction sur une date

Consigne : Lister les élèves nés après le 1er janvier 2007 (strictement postérieurs).

```
SELECT nom, prenom, dateNaiss
FROM ELEVE
WHERE dateNaiss > '2007-01-01';
```

Résultat :

nom	prenom	dateNaiss
ETOUNDI	Marie	2007-11-23
KAMGA	Grace	2008-01-30
NGONO	Paul	2007-09-05

MBARGA (2006), FOPA (2006), BELLA (2006) sont exclus.

5.5 Combinaison de conditions : AND, OR et priorités

Pour exprimer des critères complexes, on combine des conditions simples avec les opérateurs logiques AND (ET logique) et OR (OU logique).

PROPRIÉTÉ Priorité des opérateurs logiques

En SQL, comme en algorithmique et en mathématiques : AND est prioritaire sur OR.

- A AND B OR C s'interprète (A AND B) OR C.
- A OR B AND C s'interprète A OR (B AND C).

Règle d'or : Utilisez toujours des parenthèses pour lever toute ambiguïté et rendre votre intention explicite. Cela évite des bugs subtils et facilite la relecture.

ATTENTION Piège classique : AND vs OR sur le même attribut

- WHERE codeClasse = 'TC1' AND codeClasse = 'TD2' → JAMAIS VRAI. Une même ligne ne peut pas avoir deux valeurs différentes pour la même colonne en même temps. Résultat : 0 lignes.
- WHERE codeClasse = 'TC1' OR codeClasse = 'TD2' → VRAI pour les lignes qui valent 'TC1' OU 'TD2'. C'est la bonne écriture pour « élèves de TC1 ou TD2 ».

EXEMPLE Combinaison AND : restriction multiple

Consigne : Élèves de la classe TC1 ET nés après le 1er janvier 2007.

```
SELECT nom, prenom, dateNaiss, codeClasse
FROM ELEVE
WHERE codeClasse = 'TC1'
      AND dateNaiss > '2007-01-01';
```

Résultat :

nom	prenom	dateNaiss	codeClasse
ETOUNDI	Marie	2007-11-23	TC1
KAMGA	Grace	2008-01-30	TC1

Jean (E001) est en TC1 mais né en 2006 → exclu.

EXEMPLE Combinaison OR avec parenthèses

Consigne : Élèves de classe TC1 OU TD2, ET (né après 2007-01-01 OU sexe féminin).

```
SELECT nom, prenom, sexe, dateNaiss, codeClasse
FROM ELEVE
WHERE (codeClasse = 'TC1' OR codeClasse = 'TD2')
      AND (dateNaiss > '2007-01-01' OR sexe = 'F');
```

Analyse ligne par ligne :

matricule	nom	Classe OK?	(Date>2007 OU F) OK?	GLOBAL
E001	MBARGA	Oui (TC1)	Non (2006, M)	Non
E002	ETOUNDI	Oui (TC1)	Oui (F)	Oui
E003	FOPA	Oui (TD2)	Non (2006, M)	Non
E004	KAMGA	Oui (TC1)	Oui (2008, F)	Oui
E005	NGONO	Oui (TD2)	Oui (2007)	Oui
E006	BELLA	Non (TE3)	Oui (F)	Non (classe exclue)

Résultat final :

nom	prenom	sexe	dateNaiss	codeClasse
ETOUNDI	Marie	F	2007-11-23	TC1
KAMGA	Grace	F	2008-01-30	TC1
NGONO	Paul	M	2007-09-05	TD2

5.6 Élimination des doublons : DISTINCT

Par défaut, **SELECT** renvoie **toutes** les lignes qui satisfont la condition, même si plusieurs lignes sont identiques sur les colonnes projetées. Le mot-clé **DISTINCT** (placé **immédiatement après SELECT**) demande au SGBD d'éliminer les lignes dupliquées dans le **résultat final**.

PROPRIÉTÉ Syntaxe et portée de DISTINCT

```
SELECT DISTINCT liste_attributs
FROM table
WHERE condition;
```

- **DISTINCT** s'applique à l'ensemble de la ligne de résultat (la combinaison de toutes les colonnes listées).
- **SELECT DISTINCT** codeClasse, sexe FROM ELEVE : élimine les doublons sur le couple (classe, sexe).
- **SELECT DISTINCT** codeClasse FROM ELEVE : élimine les doublons sur la seule colonne classe.

ATTENTION Position de DISTINCT

DISTINCT s'écrit une seule fois, juste après **SELECT**.

- `SELECT DISTINCT nom, prenom FROM ...` ✓Correct.
- `SELECT nom, DISTINCT prenom FROM ...` Erreur de syntaxe.
- `SELECT DISTINCT nom, DISTINCT prenom FROM ...` Erreur de syntaxe.

EXEMPLE DISTINCT sur une colonne

Consigne : Quelles sont les classes représentées dans la table ? (Liste sans doublons).

```
SELECT DISTINCT codeClasse  
FROM ELEVE;
```

Résultat :

codeClasse
TC1
TD2
TE3

3 lignes au lieu de 6. L'ordre n'est pas garanti sans ORDER BY (non au programme).

EXEMPLE DISTINCT sur plusieurs colonnes

Consigne : Liste des couples (classe, sexe) existants.

```
SELECT DISTINCT codeClasse, sexe  
FROM ELEVE;
```

Résultat :

codeClasse	sexe
TC1	M
TC1	F
TD2	M
TD2	F
TE3	F

5 couples distincts. Le couple (TD2, F) n'existe pas dans nos données.

5.7 Méthode pas à pas pour construire un SELECT

MÉTHODE Construire une requête SELECT correcte

Suivez ces 4 étapes dans l'ordre :

1. Identifier la table source → Écrire `FROM nom_table`. (C'est la base, on ne peut pas sélectionner sans savoir où regarder).
2. Définir le filtre sur les lignes → Écrire la clause `WHERE condition`. Traduire l'énoncé français en booléen (attention aux AND/OR, aux parenthèses, aux guillemets simples pour chaînes/dates). Testez mentalement : « Cette ligne passe-t-elle ? ».
3. Choisir les colonnes à afficher → Écrire la liste après `SELECT` (noms exacts, séparés par des virgules, ou *). Pensez à `DISTINCT` si l'énoncé demande « la liste des... » ou « les différentes valeurs de... ».
4. Relire et valider : Vérifier l'orthographe des noms de table/colonnes, la cohérence des types (comparer date avec date, nombre avec nombre), la priorité AND/OR.

EXERCICE RÉSOLU 5 Requête complète combinée

Énoncé : Écrire la requête SQL qui affiche le nom, prénom et dateNaiss des élèves filles de la classe TC1 nées en 2007 ou 2008.

Solution détaillée :

1. `FROM ELEVE`
2. Condition : `sexe = 'F' ET codeClasse = 'TC1' ET (dateNaiss >= '2007-01-01' ET dateNaiss <= '2008-12-31')`. On peut simplifier la condition sur l'année avec `YEAR(dateNaiss) IN (2007, 2008)` mais `YEAR()` n'est pas au programme. On utilise les comparaisons de dates.
3. Colonnes : `nom, prénom, dateNaiss`.
4. Pas de doublons demandés → pas de `DISTINCT`.

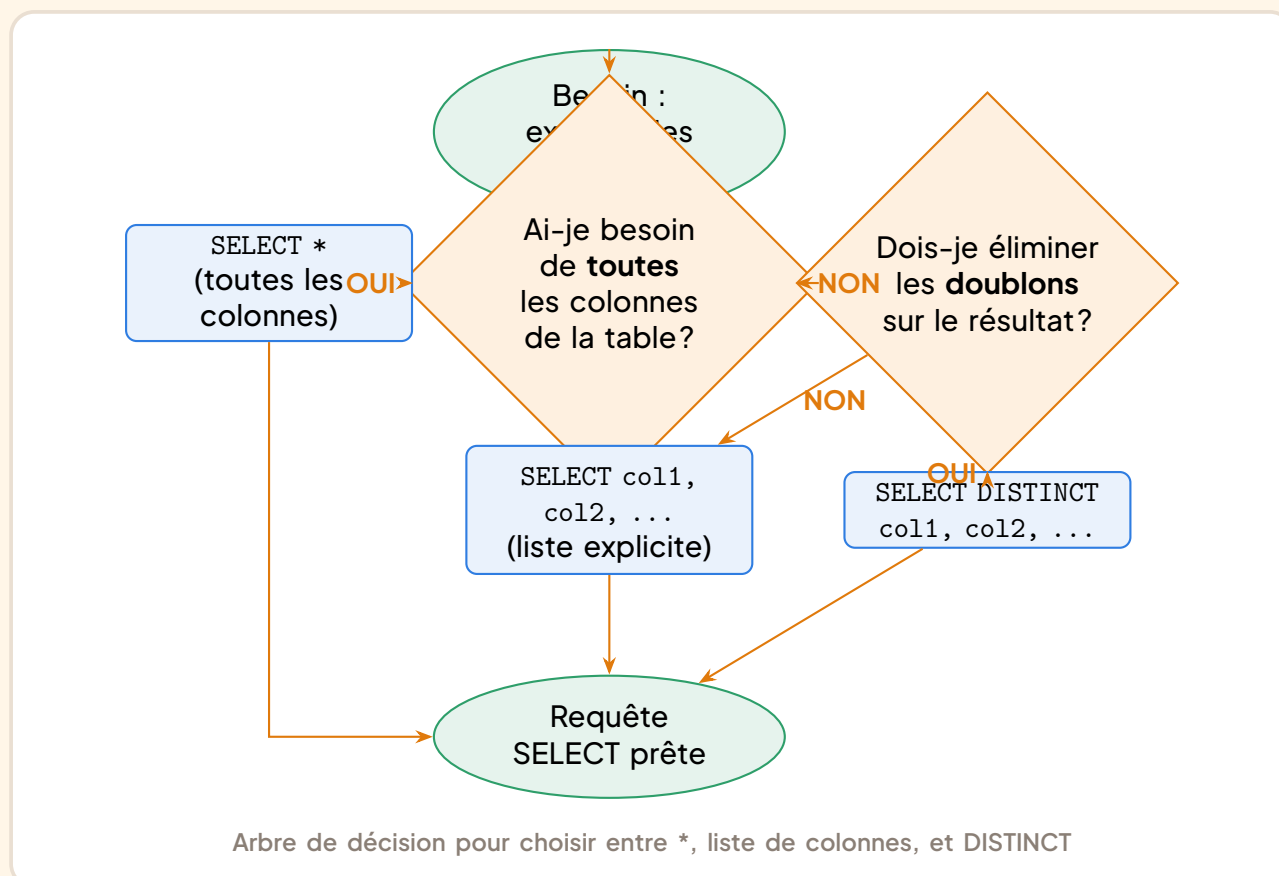
```
SELECT nom, prénom, dateNaiss
FROM ELEVE
WHERE sexe = 'F'
      AND codeClasse = 'TC1'
      AND dateNaiss >= '2007-01-01'
      AND dateNaiss <= '2008-12-31';
```

Vérification sur nos données :

- E002 ETOUNDI Marie F 2007-11-23 TC1 → OUI
- E004 KAMGA Grace F 2008-01-30 TC1 → OUI
- E001 MBARGA Jean M → Non (sexe)

- E006 BELLA Anne F 2006-12-01 TE3 → Non (classe, date)
Résultat attendu : 2 lignes (ETOUNDI, KAMGA).

5.8 Arbre de décision : comment choisir sa clause SELECT ?



5.9 Mise en œuvre graphique sous Access (Grille QBE)

Le programme exige de savoir créer une requête en mode graphique sous Access (Query By Example). C'est un complément très utile pour le Bac pratique.

EXPÉRIENCE

Créer une requête SELECT sous Access (Mode Création)

Objectif : Obtenir la liste des élèves de TC1 nés après 2007-01-01 (nom, prénom, date de naissance).

Matériel : MS Access (ou LibreOffice Base), base `Ecole.accdb` contenant la table `ELEVE`.

Manipulation pas à pas :

1. Onglet Créer → groupe Requêtes → cliquer Conception de requête.
2. Dans la boîte Afficher la table, double-cliquer sur `ELEVE` → Fermer.

3. Dans la grille QBE (bas de la fenêtre) :

- Colonne 1 : Ligne Champ → choisir `nom` (liste déroulante). Ligne Visible → cochée.
- Colonne 2 : Champ → `prenom`. Visible → cochée.
- Colonne 3 : Champ → `dateNaiss`. Visible → cochée.
- Colonne 4 : Champ → `codeClasse`. Visible → décochée (critère seulement). Ligne Critère → taper "TC1" (Access ajoute les guillemets doubles automatiquement pour le texte).
- Colonne 5 : Champ → `dateNaiss` (ou une colonne vide si on réutilise la 3). Ligne Critère → taper `>#2007-01-01#` (Access utilise # pour les dates).

4. Pour DISTINCT : Menu Conception → groupe Résultats → cocher Valeurs distinctes (icône sigma Σ avec lignes distinctes).

5. Exécuter : Onglet Conception → Exécuter (icône ! point d'exclamation rouge) ou basculer en Mode Feuille de données.

6. Basculer en Mode SQL (Vue → Mode SQL) pour voir le code généré :

```
SELECT ELEVE.nom, ELEVE.prenom, ELEVE.dateNaiss
FROM ELEVE
WHERE (((ELEVE.codeClasse)="TC1") AND ((ELEVE.dateNaiss)>#2007-01-01#));
```

Observations :

- Access préfixe les colonnes par le nom de table (`ELEVE.nom`) : bonne pratique pour éviter l'ambiguïté (indispensable en jointure).
- Les parenthèses sont abondantes : Access sécurise la priorité.
- Les chaînes sont entre " ", les dates entre # #. En SQL standard (MySQL), ce sont des guillemets simples ' ' pour les deux.

5.10 Synthèse et points de vigilance pour le Bac

À RETENIR

Check-list SELECT pour l'examen

- Syntaxe de base : `SELECT ... FROM ... WHERE ... ;` (point-virgule final).
- Guillemets : Simples 'valeur' pour chaînes et dates en SQL standard (MySQL). Doubles "valeur" en Access.
- Format date : 'AAAA-MM-JJ' (ISO 8601) en SQL standard.
- AND / OR : AND prioritaire. Toujours parenthéser les OR mélangés à des AND.
- DISTINCT : Juste après SELECT. Porte sur toute la ligne résultat.

- * vs Liste : Préférer la liste explicite (projection), sauf exploration rapide.
- Alias : Non au programme pour le SELECT simple (utile pour jointure, voir section 6).

EXERCICE 5 Entraînement SELECT

Soit la table PRODUIT (codeP, libelle, prixU, stock, codeCat) avec données :

codeP	libelle	prixU	stock	codeCat
P1	Stylo Bille	100	500	PAP
P2	Cahier 100p	500	200	PAP
P3	Clé USB 16Go	3500	50	INF
P4	Souris Optique	4500	30	INF
P5	Ramette A4	2500	100	PAP
P6	Disque Dur 1To	35000	10	INF

Écrire les requêtes SQL pour :

1. Afficher tous les produits de la catégorie 'PAP'.
2. Afficher le libellé et le prix des produits dont le stock est inférieur à 100.
3. Afficher les différentes catégories présentes.
4. Afficher les produits 'INF' OU dont le prix est supérieur à 3000.
5. Afficher le libellé et le stock des produits 'PAP' ET (stock > 150 OU prix < 300).

CORRIGÉ Exercice Entraînement SELECT

1. `SELECT * FROM PRODUIT WHERE codeCat = 'PAP';`
2. `SELECT libelle, prixU FROM PRODUIT WHERE stock < 100;`
3. `SELECT DISTINCT codeCat FROM PRODUIT;` (Résultat : PAP, INF)
4. `SELECT * FROM PRODUIT WHERE codeCat = 'INF' OR prixU > 3000;` (P3, P4, P6)
5. `SELECT libelle, stock FROM PRODUIT WHERE codeCat = 'PAP' AND (stock > 150 OR prixU < 300);` Analyse : P1 (stock 500 > 150) → Oui. P2 (stock 200 > 150) → Oui. P5 (stock 100 > 150 ET prix 2500 < 300) → Non. Résultat : P1, P2.

LESAVAIS-TU ? SQL : Un standard qui a 50 ans

Le langage SQL (Structured Query Language) est né en 1974 chez IBM (projet System R) sous le nom SEQUEL. Il a été standardisé par l'ANSI en 1986 (SQL-86) puis l'ISO en 1987. Ce qui est remarquable, c'est sa longévité : les requêtes `SELECT`, `INSERT`, `UPDATE`, `DELETE` que vous apprenez aujourd'hui en Terminale au Cameroun sont identiques à celles qu'écrivent les ingénieurs de Google, Facebook, ou les administrateurs de la base de données du Ministère de l'Éducation de Base (MINEDUB) pour gérer les millions d'élèves du pays. La puissance du déclaratif : vous dites quoi faire, le SGBD décide comment le faire (optimiseur de requêtes).

6 JOINTURE ENTRE DEUX TABLES ET MISE EN ŒUVRE PRATIQUE

Après avoir maîtrisé l'interrogation d'une seule table avec `SELECT`, nous abordons maintenant une situation bien plus fréquente dans la vie réelle : l'information que l'on cherche est éclatée sur plusieurs tables. Rappelons-nous notre modélisation (MCD/MLD) : pour éviter la redondance, nous avons séparé les élèves de leurs classes, les commandes de leurs clients, les livres de leurs auteurs. Pour retrouver une information complète — « le nom de l'élève **et** le libellé de sa classe » —, il faut reconnecter ces tables au moment de la requête. C'est précisément le rôle de la **jointure**.

6.1 Principe de la jointure : réconcilier les tables par la clé étrangère

Dans un modèle relationnel normalisé, une relation (table) `ELEVE` possède une **clé étrangère** `codeClasse` qui référence la **clé primaire** `codeClasse` de la table `CLASSE`. Physiquement, les deux tables sont stockées séparément. Logiquement, une ligne d'`ELEVE` « appartient » à une et une seule ligne de `CLASSE` grâce à l'égalité de ces deux colonnes.

DÉFINITION Jointure (équi-jointure naturelle sur clé étrangère / clé primaire)

Une jointure entre deux tables `T1` et `T2` consiste à construire un ensemble de lignes résultat en associant chaque ligne de `T1` aux lignes de `T2` qui satisfont une condition de jointure, le plus souvent l'égalité entre la clé étrangère de l'une et la clé primaire de l'autre :

$$T1.cleEtrangere = T2.clePrimaire$$

On ne conserve dans le résultat que les attributs utiles (projection) et éventuellement on filtre davantage (restriction).

6.2 Syntaxe SQL au programme : jointure dans la clause WHERE

Le programme officiel de Terminale C/D/E impose la syntaxe « ancienne » (SQL-89), encore très utilisée pour sa simplicité pédagogique, où la jointure s'écrit dans la clause `WHERE` aux côtés des éventuelles restrictions.

PROPRIÉTÉ Syntaxe de la jointure à deux tables (programme officiel)

```
SELECT t1.attribut1, t2.attribut2, ...  
FROM table1 t1, table2 t2  
WHERE t1.colonneCommune = t2.colonneCommune  
[AND conditionDeRestriction];
```

- `table1 t1, table2 t2` : les alias (noms courts) donnés aux tables. Ils sont obligatoires dès qu'un nom d'attribut existe dans les deux tables (ex. `codeClasse`) pour lever l'ambiguïté.
- `t1.colonneCommune = t2.colonneCommune` : la condition de jointure (égalité FK = PK).
- `AND conditionDeRestriction` : filtre supplémentaire (ex. ne garder que la classe 'TC1').

À RETENIR Rôle des alias

Les alias (`e` pour `ELEVE`, `c` pour `CLASSE`) servent à deux fins :

1. Raccourcir l'écriture : `e.nom` au lieu de `ELEVE.nom`.
2. Lever l'ambiguïté : si les deux tables ont une colonne `codeClasse`, écrire seulement `codeClasse` dans le `SELECT` ou le `WHERE` provoquerait une erreur « Column 'codeClasse' in field list is ambiguous ». Il faut toujours préfixer : `e.codeClasse` ou `c.codeClasse`.

6.3 Méthode pas à pas pour écrire une jointure correcte

MÉTHODE Écrire une requête de jointure entre deux tables

1. Identifier les deux tables concernées par la question (ex. `ELEVE` et `CLASSE`).
2. Trouver la colonne commune : la clé étrangère dans la table « faible » (`ELEVE.codeClasse`) et la clé primaire dans la table « forte » (`CLASSE.codeClasse`).
3. Poser des alias courts et mnémotechniques : `ELEVE e, CLASSE c`.
4. Écrire la condition de jointure dans le `WHERE` : `e.codeClasse = c.codeClasse`.
5. Préfixer tous les attributs du `SELECT` et du `WHERE` par leur alias (`e.nom, c.libelle, c.libelle = 'TC1'`).

6. Ajouter les restrictions métier avec AND si nécessaire.

6.4 Exemple complet : afficher l'élève et sa classe

Reprenons notre base ETABLISSEMENT avec les tables :

- CLASSE(codeClasse PK, libelle, effectifMax)
- ELEVE(matricule PK, nom, prenom, dateNaiss, codeClasse FK → CLASSE)

EXEMPLE Jointure simple : liste des élèves avec le libellé de leur classe

Question : Afficher le nom, le prénom de chaque élève et le libellé de sa classe.

Analyse : Deux tables (ELEVE, CLASSE), colonne commune codeClasse. On veut nom, prenom (dans ELEVE) et libelle (dans CLASSE).

Requête SQL :

```
SELECT e.nom, e.prenom, c.libelle
FROM ELEVE e, CLASSE c
WHERE e.codeClasse = c.codeClasse;
```

Explication ligne par ligne :

- SELECT e.nom, e.prenom, c.libelle : on projette trois colonnes, chacune préfixée par son alias.
- FROM ELEVE e, CLASSE c : on déclare les deux sources avec leurs alias.
- WHERE e.codeClasse = c.codeClasse : c'est le filtre de jointure. Pour chaque élève e, le SGBD cherche la (unique) classe c dont le codeClasse est égal. Les lignes qui ne trouvent pas de correspondance (élève sans classe ou classe sans élève) sont exclues (jointure interne).

Résultat attendu (extrait) :

nom	prenom	libelle
MBARGA	Jean	Terminale C1
ESSOMBA	Marie	Terminale C1
FOUDA	Paul	Terminale D2

6.5 Jointure combinée à une restriction (AND)

Très souvent, on ne veut pas toutes les lignes jointes, mais un sous-ensemble.

EXEMPLE Jointure + restriction : élèves d'une classe donnée

Question : Afficher le nom et le prénom des élèves de la classe dont le libellé est 'Terminale C1', avec le libellé de la classe.

Requête SQL :

```
SELECT e.nom, e.prenom, c.libelle
FROM ELEVE e, CLASSE c
WHERE e.codeClasse = c.codeClasse
      AND c.libelle = 'Terminale C1';
```

Observation : La condition de jointure (`e.codeClasse = c.codeClasse`) et la condition de restriction (`c.libelle = 'Terminale C1'`) sont simplement combinées par **AND** dans le même **WHERE**. L'ordre n'a pas d'importance pour le résultat, mais placer la jointure en premier est une bonne habitude de lisibilité.

6.6 Le piège majeur : l'oubli de la condition de jointure → produit cartésien

Si vous oubliez la ligne `WHERE e.codeClasse = c.codeClasse`, le SGBD n'a plus aucune instruction pour associer les lignes. Il effectue alors le **produit cartésien** (ou jointure cartésienne) : chaque ligne de la première table est combinée avec chaque ligne de la seconde.

ATTENTION Erreur fatale : produit cartésien involontaire

```
-- ERREUR : condition de jointure oubliée !
SELECT e.nom, c.libelle
FROM ELEVE e, CLASSE c;
-- (pas de WHERE, ou WHERE sans égalité sur les clés)
```

Conséquences :

- Si **ELEVE** a 500 lignes et **CLASSE** 20 lignes → 10 000 lignes dans le résultat.
- Le résultat est sémantiquement faux : un même élève apparaît 20 fois, associé à des classes qui ne sont pas la sienne.
- Temps d'exécution explosif, saturation mémoire possible sur de grosses tables.

Règle d'or : Pour n tables jointes, il faut au minimum $n - 1$ conditions d'égalité dans le **WHERE** (une par lien FK-PK du graphe de jointure).

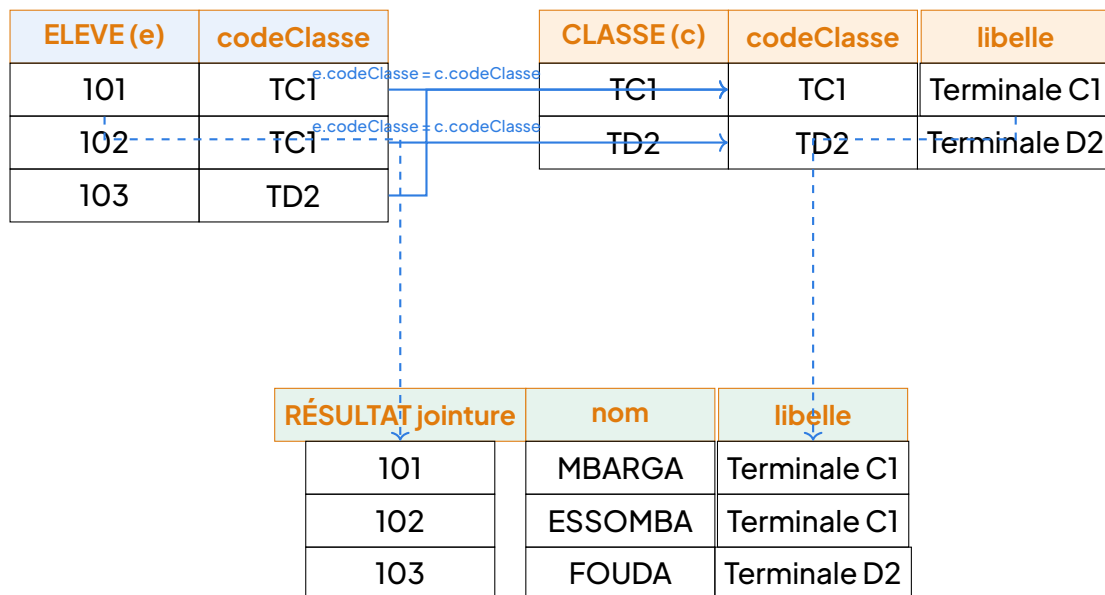


Schéma de la jointure interne : seules les lignes dont les `codeClasse` correspondent sont combinées (3 lignes résultat pour 3 élèves).

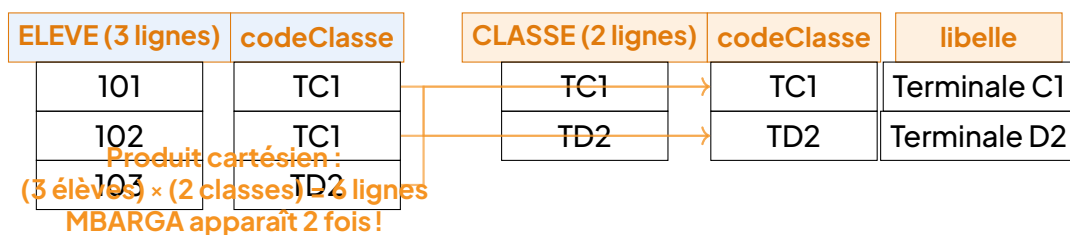


Illustration du produit cartésien : sans condition de jointure, chaque élève est associé à toutes les classes. Résultat faux et volumineux.

6.7 Mise en œuvre pratique : création et exploitation en console MySQL

Le programme exige de savoir créer la base et les tables en **mode console** (ligne de commande MySQL) puis d'y insérer et sélectionner des données. Voici la séquence complète, testée sur un serveur MySQL/MariaDB standard (ex. XAMPP, WAMP, ou serveur Linux du lycée).

EXPÉRIENCE**TP guidé : Base ETABLISSEMENT en console MySQL**

Objectif : Créer la base, les deux tables liées par une clé étrangère, insérer des données, faire une jointure. Matériel : Accès à un terminal (Invite de commandes Windows / Terminal Linux) avec `mysql` client installé.

Étapes :

1. Ouvrir le client MySQL : `mysql -u root -p` (Entrer le mot de passe, souvent vide en local XAMPP).

2. Créer la base :

```
CREATE DATABASE ETABLISSEMENT CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
```

3. Sélectionner la base :

```
USE ETABLISSEMENT;
```

4. Créer la table parente (CLASSE) d'abord (pas de FK sortante) :

```
CREATE TABLE CLASSE (  
  codeClasse VARCHAR(10) PRIMARY KEY,  
  libelle VARCHAR(50) NOT NULL,  
  effectifMax INT UNSIGNED DEFAULT 60  
) ENGINE=InnoDB;
```

5. Créer la table fille (ELEVE) avec la FK :

```
CREATE TABLE ELEVE (  
  matricule INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  nom VARCHAR(50) NOT NULL,  
  prenom VARCHAR(50) NOT NULL,  
  dateNaiss DATE,  
  codeClasse VARCHAR(10) NOT NULL,  
  CONSTRAINT fk_eleve_classe  
    FOREIGN KEY (codeClasse) REFERENCES CLASSE(codeClasse)  
    ON UPDATE CASCADE ON DELETE RESTRICT  
) ENGINE=InnoDB;
```

6. Insérer des classes :

```
INSERT INTO CLASSE (codeClasse, libelle, effectifMax) VALUES  
( 'TC1', 'Terminale C1', 55),  
( 'TD2', 'Terminale D2', 50),  
( 'TE1', 'Terminale E1', 45);
```

7. Insérer des élèves (la FK doit exister dans CLASSE) :

```
INSERT INTO ELEVE (nom, prenom, dateNaiss, codeClasse) VALUES  
( 'MBARGA', 'Jean', '2006-03-12', 'TC1'),  
( 'ESSOMBA', 'Marie', '2005-11-25', 'TC1'),  
( 'FOUDA', 'Paul', '2006-07-01', 'TD2'),  
( 'ATANGANA', 'Grace', '2005-09-18', 'TE1');
```

8. Tester la jointure :

```
SELECT e.nom, e.prenom, c.libelle  
FROM ELEVE e, CLASSE c  
WHERE e.codeClasse = c.codeClasse  
AND c.libelle = 'Terminale C1';
```

9. Quitter : EXIT;

Observations attendues :

- L'étape 5 réussit car ENGINE=InnoDB (moteur par défaut récent) supporte les FK. MyISAM les ignorerait.
- L'étape 7 échouerait si on mettait codeClasse = 'TX9' (erreur 1452 : Cannot add or update a child row).
- L'étape 8 affiche exactement les deux élèves de TC1.

```
mysql> CREATE DATABASE ETABLISSEMENT;
Query OK, 1 row affected (0.01 sec)

mysql> USE ETABLISSEMENT;
Database changed

mysql> CREATE TABLE CLASSE (...) ENGINE=InnoDB;
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE ELEVE (...
FOREIGN KEY (codeClasse) REFERENCES CLASSE(codeClasse));
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO CLASSE VALUES ('TC1','Terminale C1',55),...;
Query OK, 3 rows affected (0.01 sec)

mysql> INSERT INTO ELEVE (nom,prenom,dateNaiss,codeClasse) VALUES ...;
Query OK, 4 rows affected (0.01 sec)

mysql> SELECT e.nom, c.libelle FROM ELEVE e, CLASSE c
WHERE e.codeClasse = c.codeClasse;
+-----+-----+
| nom | libelle |
+-----+-----+
| MBARGA | Terminale C1 |
| ESSOMBA | Terminale C1 |
| FOUDA | Terminale D2 |
| ATANGANA | Terminale E1 |
+-----+-----+
4 rows in set (0.00 sec)

mysql> EXIT;
Bye
```

Capture stylisée d'une session console MySQL : création de la base, des tables (avec FK), insertion et jointure.

6.8 Équivalent sous Microsoft Access (mode graphique)

Le programme mentionne aussi l'interface graphique d'Access. La logique relationnelle est identique, mais les manipulations se font à la souris.

MÉTHODE Créer la base ETABLISSEMENT sous Access

1. Nouvelle base : Fichier → Nouveau → Base de données vide → Nommer ETABLISSEMENT.accdb.
2. Créer la table CLASSE (mode Création) :
 - Onglet Création → Table (mode Création).
 - Champs : codeClasse (Texte, Taille 10) → Clic droit → Clé primaire ; libelle (Texte, 50, Requis=Oui) ; effectifMax (Entier long).

- Enregistrer sous CLASSE.
3. Créer la table ELEVE (mode Création) :
 4. Champs : `matricule` (Numérique, Type Entier long, Index=Oui (Sans doublons) → Clé primaire); `nom`, `prenom` (Texte, Requis=Oui); `dateNaiss` (Date/Heure); `codeClasse` (Texte, Taille 10, Requis=Oui).
 5. Enregistrer sous ELEVE.
 6. Définir la relation (Fenêtre Relations) :
 - Onglet Outils de base de données → Relations.
 - Ajouter CLASSE et ELEVE → Fermer.
 - Glisser `codeClasse` de CLASSE sur `codeClasse` de ELEVE.
 - Cocher Intégrité référentielle, Cascade mise à jour ; laisser Cascade suppression décoché (suppression restreinte).
 - Créer. Le trait « 1 » vers « ∞ » apparaît.
 7. Saisir des données : Ouvrir chaque table en mode Feuille de données, remplir.
 8. Faire la jointure (Requête en mode SQL) :
 - Onglet Création → Requête (mode SQL).
 - Coller la requête SQL identique à MySQL (Access comprend cette syntaxe).
 - Exécuter (!) → Résultat en grille.
 9. Alternative : Requête en mode Grille (QBE – Query By Example) :
 - Créer → Requête (mode Création) → Ajouter ELEVE, CLASSE.
 - La ligne de liaison (jointure) existe déjà grâce à la fenêtre Relations.
 - Double-cliquer `nom`, `prenom` (ELEVE) et `libelle` (CLASSE) dans la grille.
 - Critère sur `libelle` : "Terminale C1".
 - Exécuter.

PROPRIÉTÉ Lien entre contrainte REFERENCES et condition de jointure (admise)

La contrainte `FOREIGN KEY ... REFERENCES` posée lors du `CREATE TABLE` (ou via la fenêtre Relations d'Access) garantit l'intégrité référentielle à l'insertion/modification : on ne peut pas mettre un `codeClasse` inexistant dans ELEVE. La condition de jointure `WHERE e.codeClasse = c.codeClasse` dans le `SELECT` exploite cette intégrité pour reconstruire l'information complète à la lecture. Si l'intégrité n'est pas garantie au niveau du SGBD (ex. tables MyISAM, ou FK désactivée), la jointure fonctionne quand même mais peut retour-

ner des lignes « orphelines » (élève sans classe correspondante) si on utilise une jointure externe (LEFT JOIN), hors programme. Au programme, la jointure interne suppose l'intégrité.

6.9 Exercice résolu type Bac

EXERCICE RÉSOLU 6 Jointure sur base GESTION_COMMANDES

Énoncé : On considère la base de données d'un cybercafé CYBER_DOUALA avec deux tables :

- CLIENT(idClient PK, nom, prenom, quartier)
- COMMANDE(numCmd PK, dateCmd, montant, idClient FK → CLIENT)

Écrire la requête SQL qui affiche, pour chaque commande de plus de 5000 FCFA, le numéro de commande, la date, le montant, le nom et le prénom du client concerné. Trier par montant décroissant.

Solution détaillée :

1. Tables : COMMANDE (c) et CLIENT (cl). Colonne commune : idClient.
2. Alias : c pour COMMANDE, cl pour CLIENT.
3. Jointure : c.idClient = cl.idClient.
4. Restriction : c.montant > 5000.
5. Projection : c.numCmd, c.dateCmd, c.montant, cl.nom, cl.prenom.
6. Tri : ORDER BY c.montant DESC.

```
SELECT c.numCmd, c.dateCmd, c.montant, cl.nom, cl.prenom
FROM COMMANDE c, CLIENT cl
WHERE c.idClient = cl.idClient
AND c.montant > 5000
ORDER BY c.montant DESC;
```

Vérification : Tous les attributs sont préfixés. La jointure est présente. La restriction métier est ajoutée par AND. Le tri est explicite.

6.10 Exercices d'entraînement

EXERCICE 6 Jointure bibliothèque scolaire

La base BIBLIO_LYCEE contient : AUTEUR(idAuteur PK, nomAuteur, pays) et LIVRE(isbn

PK, titre, annee, idAuteur FK → AUTEUR).

1. Écrire la requête qui affiche le titre de chaque livre et le nom de son auteur.
2. Modifier pour n'afficher que les livres publiés après 2010, avec le pays de l'auteur.
3. Que se passe-t-il si un livre a un idAuteur qui n'existe pas dans AUTEUR (base corrompue) et qu'on fait la jointure interne ci-dessus ?

EXERCICE 7 Jointure + agrégat (révision)

Sur la base ETABLISSEMENT (ELEVE, CLASSE), écrire la requête qui affiche, pour chaque classe, son libellé et le nombre d'élèves qu'elle contient. Ordonner par effectif décroissant. Indication : GROUP BY c.libelle et COUNT(e.matricule).

CORRIGÉ Exercice 1

1. `SELECT l.titre, a.nomAuteur FROM LIVRE l, AUTEUR a WHERE l.idAuteur = a.idAuteur;`
2. `SELECT l.titre, a.nomAuteur, a.pays FROM LIVRE l, AUTEUR a WHERE l.idAuteur = a.idAuteur AND l.annee > 2010;`
3. Le livre « orphelin » disparaît du résultat. La jointure interne ne conserve que les paires qui satisfont l'égalité. C'est pourquoi l'intégrité référentielle (FK) est cruciale : elle empêche l'existence de tels orphelins.

CORRIGÉ Exercice 2

```
SELECT c.libelle, COUNT(e.matricule) AS effectif
FROM ELEVE e, CLASSE c
WHERE e.codeClasse = c.codeClasse
GROUP BY c.libelle
ORDER BY effectif DESC;
```

Note : COUNT(e.matricule) compte les lignes par groupe. GROUP BY c.libelle suppose que libelle est unique (clé candidate), sinon grouper par c.codeClasse et sélectionner c.libelle (MySQL autorise cela si ONLY_FULL_GROUP_BY est désactivé, mais le standard SQL exige de grouper par tous les attributs non agrégés du SELECT).

LESAVAIS-TU ? Le savais-tu ? : La jointure, star du Baccalauréat

Depuis la réforme 2018 du programme MINESEC, la question sur la jointure à deux tables avec alias est systématiquement présente à l'épreuve pratique (TP) ou écrite de l'Informatique en Terminale C, D, E. Elle tombe sous l'une de ces formes :

- « Écrire la requête qui affiche ... » (code SQL à produire).
- « Compléter la requête suivante : `SELECT ...FROM ...WHERE _____` » (trouver la condition de jointure).
- « Expliquer l'erreur dans cette requête : `SELECT nom, libelle FROM ELEVE, CLASSE;` » (produit cartésien).

Astuce de champion : Sur votre brouillon, dessinez toujours les deux tables avec leurs colonnes, entourez la clé étrangère et la clé primaire, tracez une flèche, écrivez l'alias au-dessus de chaque table. La requête s'écrit alors toute seule. Ne sautez jamais cette étape, même si l'exercice vous semble simple. C'est la garantie de ne pas oublier le `WHERE` ou de mélanger les alias.

7 ACTIVITÉ D'INTÉGRATION

8 IMPLÉMENTATION D'UNE BASE DE DONNÉES

À RETENIR Objectifs de l'activité

Cette activité d'intégration vise à mobiliser l'ensemble des savoir-faire de la leçon « Implémentation d'une base de données » dans une situation professionnelle réaliste. À l'issue de ce travail, vous devez être capable de :

- Concevoir la structure d'une base de données relationnelle simple (deux tables) en respectant les règles d'intégrité (clés primaires, clé étrangère, **NOT NULL**) ;
- Rédiger les requêtes **SQL** de définition des données (**CREATE DATABASE**, **CREATE TABLE**, **ALTER**, **DROP**) ;
- Manipuler les données avec les ordres **INSERT**, **UPDATE**, **DELETE** ;
- Interroger la base par des requêtes **SELECT** mono-table (filtrage, tri, sélection de colonnes) et bi-table (jointure équijointure avec alias) ;
- Exécuter un script **SQL** complet sous **MySQL** (console) ou **Access** (mode graphique) et en interpréter les résultats.

8.1 Situation

Le proviseur du **Lycée Bilingue de Nkolbisson** (Yaoundé) souhaite informatiser la gestion des conseils de classe du premier trimestre. Il vous confie, en tant qu'administrateur de la base de données de l'établissement, la mise en place d'une application de suivi des élèves de Terminale. Pour cette phase pilote, ne sont concernées que trois classes : **TC1** (Terminale C1), **TD1** (Terminale D1) et **TE1** (Terminale E1).

Vous disposez de la fiche de renseignements suivante, établie par le secrétariat :

Table **CLASSE** (3 enregistrements)

CodeClasse	Libelle	EffectifPrevu
TC1	Terminale C1	30
TD1	Terminale D1	28
TE1	Terminale E1	25

Table **ELEVE** (8 enregistrements initiaux)

Matricule	NomComplet	DateNaissance	CodeClasse
24T001	Mbarga Jean	2006-05-12	TC1
24T002	Fouda Marie	2007-03-22	TC1
24T003	Essomba Pierre	2005-11-30	TD1
24T004	Atangana Grace	2007-08-14	TD1
24T005	Ndi Paul	2006-01-20	TE1
24T006	Owona Anne	2007-12-05	TE1
24T007	Biya Samuel	2006-09-18	TC1
24T008	Mvondo Rose	2005-04-25	TD1

Deux anomalies sont signalées par les surveillants généraux avant le conseil :

1. L'élève **Essomba Pierre** (Matricule 24T003) a été inscrit par erreur en **TD1** alors qu'il suit le cursus **Terminale C**. Il doit être rattaché à la classe **TC1**.
2. L'élève **Mvondo Rose** (Matricule 24T008) a été radiée des effectifs pour abandon scolaire ; elle doit être supprimée de la base.

Le proviseur attend trois états pour la séance du conseil :

1. La liste complète des élèves de la classe **TC1** (matricule, nom, date de naissance) triée par nom.
2. La liste des élèves nés **après le 1er janvier 2007** (toutes classes confondues) pour l'attribution des bourses d'excellence jeunes.
3. Un tableau de correspondance **Nom de l'élève – Libellé de sa classe** pour l'affichage dans le hall, obtenu par jointure des deux tables.

8.2 Consignes

Travail en groupes de 4 élèves. Chaque groupe remet un script **SQL** unique (fichier .sql ou feuille manuscrite). Les groupes dotés de poste informatique testent leur script sous **MySQL** (invite de commande `mysql>`) ou **Access** (onglet Création → Conception de requête → Mode SQL). Les autres groupes valident la syntaxe sur papier.

1. **Création de la base et des tables** : Écrire les instructions **CREATE DATABASE** (nommée **LyceeNkolbisson**) et **CREATE TABLE** pour **CLASSE** et **ELEVE**. Choisir les types adaptés (**VARCHAR**, **DATE**, **INT**, **CHAR**) et poser **toutes** les contraintes : clés primaires, clé étrangère (**REFERENCES**), **NOT NULL** sur les colonnes non nulles.
2. **Insertion des données** : Rédiger les 11 instructions **INSERT INTO** (3 pour **CLASSE**, 8 pour **ELEVE**) en respectant l'ordre des colonnes et le format de date AAAA-MM-JJ.
3. **Correction de l'erreur de saisie** : Écrire la requête **UPDATE** qui affecte l'élève 24T003 à la classe **TC1**.
4. **Suppression de l'élève radié** : Écrire la requête **DELETE** qui retire l'élève 24T008.
5. **Requêtes de sélection pour le proviseur** :

- (a) **SELECT** : élèves de **TC1** (matricule, nom, date de naissance), triés par nom.
- (b) **SELECT** : élèves nés après le 2007-01-01 (nom, date de naissance, classe).
- (c) **SELECT** avec **jointure** : afficher NomComplet (alias **Eleve**) et Libelle (alias **Classe**) pour tous les élèves restants, en utilisant des alias de table (e et c).
6. **Nettoyage (optionnel)** : Écrire l'instruction **DROP DATABASE** pour remettre l'environnement propre après la démo.

PROPRIÉTÉ Ressources syntaxiques SQL à mobiliser

- **Syntaxe CREATE TABLE** : `CREATE TABLE NomTable (Col1 Type Contraintes, Col2 Type Contraintes, ..., CONSTRAINT NomFK FOREIGN KEY (ColLocale) REFERENCES TableRef(ColRef));`
- **Syntaxe ALTER TABLE** (ajout/suppression colonne, modification type, ajout contrainte): `ALTER TABLE NomTable ADD Colonne Type Contraintes;`
`ALTER TABLE NomTable DROP COLUMN Colonne;`
`ALTER TABLE NomTable MODIFY Colonne NouveauType;`
`ALTER TABLE NomTable ADD CONSTRAINT NomFK FOREIGN KEY (ColLocale) REFERENCES TableRef(ColRef);`
- **Types courants** : **VARCHAR(n)** (chaîne variable), **CHAR(n)** (chaîne fixe), **INT** (entier), **DATE** (format YYYY-MM-DD).
- **Contraintes** : **PRIMARY KEY** (identifiant unique, non nul), **NOT NULL** (interdit la valeur NULL), **REFERENCES** (intégrité référentielle).
- **INSERT** : `INSERT INTO Table (Col1, Col2, ...) VALUES (Val1, Val2, ...);` — Les chaînes et dates entre guillemets simples.
- **UPDATE** : `UPDATE Table SET Col = NouvVal WHERE Condition;` — Jamais sans **WHERE**!
- **DELETE** : `DELETE FROM Table WHERE Condition;` — Jamais sans **WHERE**!
- **SELECT simple** : `SELECT Col1, Col2 FROM Table WHERE Condition ORDER BY Col1;`
- **Jointure équi-naturelle (syntaxe classique)** : `SELECT t1.ColA, t2.ColB FROM Table1 t1, Table2 t2 WHERE t1.ColCleEtr = t2.ColClePrim;`
- **Opérateurs de comparaison** : `=`, `<>`, `<`, `>`, `<=`, `>=`; **Logiques** : `AND`, `OR`, `NOT`.
- **DISTINCT** : élimine les doublons; `*` : toutes les colonnes.

EXERCICE RÉSOLU 7 Corrigé type de l'activité d'intégration – Base Lycée Nkolbisson

Étape 1 – Création de la base et des tables

On crée d'abord la base, puis on s'y place (USE). La table **CLASSE** est créée avant **ELEVE** car cette dernière y fait référence (clé étrangère). On utilise **CHAR(3)** pour les codes

courts de longueur fixe (TC1, TD1, TE1) et **VARCHAR** pour les libellés variables. La date de naissance est de type **DATE**.

```
-- 1. Création de la base de données
CREATE DATABASE LyceeNkolbisson;
USE LyceeNkolbisson;

-- 2. Table CLASSE (pas de clé étrangère, donc créée en premier)
CREATE TABLE CLASSE (
    CodeClasse CHAR(3) PRIMARY KEY,
    Libelle VARCHAR(50) NOT NULL,
    EffectifPrevu INT NOT NULL
);

-- 3. Table ELEVE avec clé étrangère vers CLASSE
CREATE TABLE ELEVE (
    Matricule VARCHAR(10) PRIMARY KEY,
    NomComplet VARCHAR(100) NOT NULL,
    DateNaissance DATE NOT NULL,
    CodeClasse CHAR(3) NOT NULL,
    CONSTRAINT FK_Eleve_Classe
        FOREIGN KEY (CodeClasse) REFERENCES CLASSE(CodeClasse)
);
```

Étape 2 – Insertion des 11 lignes

L'ordre d'insertion respecte l'intégrité référentielle : d'abord les 3 classes (parents), ensuite les 8 élèves (enfants). Les littéraux de type **DATE** et **VARCHAR/CHAR** sont entourés de guillemets simples '... '.

```
-- Insertion des classes
INSERT INTO CLASSE (CodeClasse, Libelle, EffectifPrevu) VALUES
('TC1', 'Terminale C1', 30),
('TD1', 'Terminale D1', 28),
('TE1', 'Terminale E1', 25);

-- Insertion des élèves (8 lignes)
INSERT INTO ELEVE (Matricule, NomComplet, DateNaissance, CodeClasse) VALUES
('24T001', 'Mbarga Jean', '2006-05-12', 'TC1'),
('24T002', 'Fouda Marie', '2007-03-22', 'TC1'),
('24T003', 'Essomba Pierre', '2005-11-30', 'TD1'), -- Erreur volontaire ici (TD1 au lieu de TC1)
('24T004', 'Atangana Grace', '2007-08-14', 'TD1'),
('24T005', 'Ndi Paul', '2006-01-20', 'TE1'),
('24T006', 'Owona Anne', '2007-12-05', 'TE1'),
('24T007', 'Biya Samuel', '2006-09-18', 'TC1'),
('24T008', 'Mvondo Rose', '2005-04-25', 'TD1');
```

Étape 3 – Correction par **UPDATE** (Essomba Pierre)

La clause **WHERE** cible uniquement le matricule 24T003. Sans **WHERE**, tous les élèves passeraient en TC1 (erreur classique).

```
UPDATE ELEVE
SET CodeClasse = 'TC1'
WHERE Matricule = '24T003';
-- Résultat attendu : 1 ligne affectée (Rows matched : 1, Changed : 1)
```

Étape 4 – Suppression par **DELETE** (Mvondo Rose)

Même vigilance : **WHERE** sur la clé primaire **Matricule** pour ne supprimer que la ligne concernée.

```
DELETE FROM ELEVE
WHERE Matricule = '24T008';
-- Résultat attendu : 1 ligne supprimée
```

Étape 5 – Requêtes de sélection pour le proviseur

Requête 5.1 : Élèves de TC1 (matricule, nom, naissance), triés par nom.

```
SELECT Matricule, NomComplet, DateNaissance
FROM ELEVE
WHERE CodeClasse = 'TC1'
ORDER BY NomComplet;
```

Résultat attendu (4 lignes après UPDATE) : Mbarga Jean, Biya Samuel, Essomba Pierre, Fouda Marie.

Requête 5.2 : Élèves nés après le 01/01/2007 (toutes classes).

```
SELECT NomComplet, DateNaissance, CodeClasse
FROM ELEVE
WHERE DateNaissance > '2007-01-01';
```

Résultat attendu (3 lignes) : Fouda Marie (2007-03-22), Atangana Grace (2007-08-14), Owona Anne (2007-12-05). Attention : l'opérateur est > (strictement après), pas >=.

Requête 5.3 : Jointure Élève – Classe avec alias (Nom élève, Libellé classe). On utilise des alias de table (e pour **ELEVE**, c pour **CLASSE**) pour préfixer les colonnes et lever toute ambiguïté. La condition de jointure est e.CodeClasse = c.CodeClasse.

```
SELECT e.NomComplet AS Eleve, c.Libelle AS Classe
FROM ELEVE e, CLASSE c
WHERE e.CodeClasse = c.CodeClasse
ORDER BY c.Libelle, e.NomComplet;
```

Résultat attendu (7 lignes, Mvondo Rose supprimée, Essomba Pierre maintenant en TC1) :

Eleve	Classe
Mbarga Jean	Terminale C1
Biya Samuel	Terminale C1
Essomba Pierre	Terminale C1
Fouda Marie	Terminale C1
Atangana Grace	Terminale D1
Ndi Paul	Terminale E1
Owona Anne	Terminale E1

Étape 6 – Nettoyage (optionnel)

Pour supprimer entièrement la base de données après la démonstration :

```
DROP DATABASE LyceeNkolbisson;
```

SOLUTION

Commentaires pédagogiques du corrigé

- Ordre des opérations DDL : La table référencée (**CLASSE**) doit exister avant la table qui la référence (**ELEVE**). Inverser l'ordre provoque une erreur `ERROR 1215 (HY000): Cannot add foreign key constraint.`
- Format de date : MySQL attend `YYYY-MM-DD` (norme ISO 8601). Écrire `'12/05/2006'` (format FR) entraîne une erreur ou une insertion `0000-00-00` selon le mode **SQL_MODE**.
- Clé étrangère : La contrainte `FK_Eleve_Classe` empêche d'insérer un élève avec un **CodeClasse** inexistant (ex. `'TX1'`) et empêche de supprimer une classe tant qu'elle a des élèves (sauf **ON DELETE CASCADE** non vu au programme).
- **UPDATE/DELETE** sans **WHERE** : Rappelons que c'est la première cause de perte de données en production. Toujours tester la condition avec un **SELECT** avant d'exécuter la modification.
- Alias dans la jointure : L'écriture `e.NomComple` est obligatoire si le nom de colonne existe dans les deux tables (ici **CodeClasse** est commun, d'où `e.CodeClasse` et `c.CodeClasse`). L'alias de colonne (`AS Eleve`) améliore la lisibilité de l'affichage final.
- Jointure vs Produit cartésien : Oublier la condition `WHERE e.CodeClasse = c.CodeClasse` produit un produit cartésien ($7 \times 3 = 21$ lignes), résultat sans sens.

À RETENIR Bilan de l'activité

Cette activité a permis de mettre en évidence, par la pratique, le cycle de vie complet d'une base de données relationnelle dans un contexte administratif camerounais concret :

- La modélisation physique (choix des types, définition des clés primaires et étrangères) conditionne la qualité et la fiabilité des données stockées. L'omission d'une contrainte **NOT NULL** ou **REFERENCES** ouvre la porte à des anomalies (élèves sans classe, classes fantômes).
- Le langage SQL se révèle être un standard puissant : les mêmes instructions s'exécutent indifféremment en console **MySQL** (serveur, multi-utilisateurs, réseau) ou en mode graphique **Access** (monoposte, fichier local `.accdb`), ce qui illustre la portabilité du standard.
- La manipulation de données (**INSERT**, **UPDATE**, **DELETE**) exige une rigueur absolue sur les clauses **WHERE** : une requête mal ciblée modifie ou efface bien plus que prévu.
- L'interrogation (**SELECT**) montre tout l'intérêt du relationnel : la jointure permet de

restituer une information « métier » (Nom + Libellé classe) qui n'existe pas physiquement dans une seule table, évitant ainsi la redondance et les incohérences.

- Enfin, le travail en groupe et la confrontation papier/console ont renforcé la compétence « Communiquer sur une solution technique » : expliquer son script, justifier un type **CHAR** vs **VARCHAR**, ou déboguer une erreur de syntaxe `ERROR 1064` sont des gestes professionnels essentiels pour le Baccalauréat et au-delà.

9 MÉTHODES ET EXERCICES RÉSOLUS

10 LEÇON 6 : IMPLÉMENTATION D'UNE BASE DE DONNÉES

10.1 Bases de données, SGBD et langage SQL

DÉFINITION Notions fondamentales

- Base de données (BD) : Ensemble structuré de données persistantes, organisées pour un accès, une gestion et une mise à jour faciles. Elle modélise une partie du monde réel (ex : gestion d'un lycée, d'un cybercafé, d'une boutique).
- SGBD (Système de Gestion de Base de Données) : Logiciel serveur permettant de créer, manipuler, sécuriser et partager des bases de données. Exemples au programme : Microsoft Access (monoposte, fichier .accdb), MySQL / MariaDB (client-serveur, standard web/entreprise).
- SQL (Structured Query Language) : Langage standardisé (ISO/ANSI) pour dialoguer avec un SGBD relationnel. Deux sous-langages principaux :
 - DDL (Data Definition Language) : CREATE, ALTER, DROP (structure).
 - DML (Data Manipulation Language) : INSERT, UPDATE, DELETE, SELECT (données).
- Requête : Instruction SQL envoyée au SGBD pour obtenir un résultat (création d'objet, modification de données, affichage de résultats).

À RETENIR Architecture trois niveaux (rappel)

Le SGBD sépare : Niveau externe (vues utilisateurs), Niveau conceptuel (schéma global, MCD/MLD), Niveau interne (stockage physique, index). L'indépendance des données permet de changer le stockage sans casser les applications.

10.2 Types de données et contraintes d'intégrité

PROPRIÉTÉ Types de données SQL au programme (Terminale C/D/E)

Type SQL	Usage / Description	Exemple littéral
INT	Entier signé (4 octets), identifiants, compteurs, âges.	123, -5
VARCHAR(<i>n</i>)	Chaîne de longueur variable (0 à <i>n</i> caractères). Efficace pour noms, adresses, emails. $n \leq 65535$.	'MBARGA', 'jean@email.cm'
CHAR(<i>n</i>)	Chaîne de longueur fixe <i>n</i> (complétée par des espaces). Codes fixes : sexe, code postal, matricule.	'M', 'E20240001'
DATE	Date calendaire (année, mois, jour). Format standard ISO : AAAA-MM-JJ.	'2006-05-12'

Note : TEXT, BLOB, DECIMAL, DATETIME existent mais ne sont pas au programme officiel.

PROPRIÉTÉ Contraintes d'intégrité (garantissent la qualité des données)

- **PRIMARY KEY (PK) :** Identifiant unique de la ligne. Non nul + Unique. Une seule PK par table (simple ou composée).
- **NOT NULL :** La colonne ne peut pas recevoir la valeur NULL (absence de valeur). Obligatoire pour les attributs identifiants ou métier indispensables.
- **UNIQUE :** Toutes les valeurs de la colonne (ou groupe de colonnes) sont distinctes. Accepte NULL (sauf MySQL où NULL ≠ NULL pour l'unicité). Ex : Email, Matricule (si pas PK).
- **FOREIGN KEY (FK) REFERENCES :** Intégrité référentielle. La valeur de la colonne enfant doit exister comme clé primaire dans la table parente (ou être NULL si autorisé). Empêche les orphelins.
- **DEFAULT valeur :** Valeur insérée automatiquement si omise dans INSERT. (Non explicitement au programme mais utile).

10.3 Requêtes de définition des données : CREATE, ALTER, DROP

MÉTHODE Créer une table avec types et contraintes (CREATE TABLE)

Objectif : Écrire une requête DDL pour créer une table structurée respectant l'intégrité.

1. Analyser le besoin : Nom table, liste colonnes, types, contraintes (PK, NN, UQ, FK).
2. Choisir les types : INT (id), VARCHAR(*n*) (texte variable), CHAR(*n*) (code fixe), DATE (date ISO).

3. Poser les contraintes : PRIMARY KEY, NOT NULL, UNIQUE, FOREIGN KEY ... REFERENCES
....
4. Rédiger la requête : Syntaxe canonique.
5. Vérifier : Parenthèses, virgules (pas après dernière colonne), noms sans espaces/accent (ou backticks `).

EXERCICE RÉSOLU 8 Création de la table ETUDIANT d'un lycée

Énoncé : Créer la table ETUDIANT :

- Matricule : CHAR(10), **identifiant unique**.
- Nom, Prenom : VARCHAR(50), **obligatoires**.
- DateNaiss : DATE, **obligatoire**.
- Sexe : CHAR(1) ('M'/'F'), **obligatoire**.
- Email : VARCHAR(100), **unique**.
- IdClasse : INT, **référence** CLASSE(IdClasse).

SOLUTION

Solution détaillée :

1. **Colonnes & Types** : Matricule CHAR(10), Nom/Prenom VARCHAR(50), DateNaiss DATE, Sexe CHAR(1), Email VARCHAR(100), IdClasse INT.
2. **Contraintes** : PK sur Matricule, NN sur Nom, Prenom, DateNaiss, Sexe, IdClasse, UQ sur Email, FK IdClasse → CLASSE(IdClasse).
3. **Requête** :

```
CREATE TABLE ETUDIANT (  
  Matricule CHAR(10) PRIMARY KEY,  
  Nom VARCHAR(50) NOT NULL,  
  Prenom VARCHAR(50) NOT NULL,  
  DateNaiss DATE NOT NULL,  
  Sexe CHAR(1) NOT NULL,  
  Email VARCHAR(100) UNIQUE,  
  IdClasse INT NOT NULL,  
  CONSTRAINT FK_Etudiant_Classe  
  FOREIGN KEY (IdClasse) REFERENCES CLASSE(IdClasse)  
);
```

Conclusion : Table créée avec PK, unicité email, FK vers CLASSE.

MÉTHODE Modifier et supprimer la structure d'une table (ALTER TABLE, DROP TABLE)

Objectif : Adapter le schéma (ajouter/supprimer colonne, modifier type/contrainte) ou supprimer la table.

1. Ajouter une colonne : `ALTER TABLE NomTable ADD NomColonne Type [Contraintes];`
2. Supprimer une colonne : `ALTER TABLE NomTable DROP COLUMN NomColonne;` (Perte de données irréversible).
3. Modifier une colonne (type/contraintes) :
 - **MySQL** : `ALTER TABLE NomTable MODIFY NomColonne NouveauType NouvellesContraintes;`
 - **Standard SQL / Access** : `ALTER TABLE NomTable ALTER COLUMN NomColonne NouveauType [Contraintes];`
4. Ajouter une contrainte (FK) : `ALTER TABLE NomTable ADD CONSTRAINT NomContrainte FOREIGN KEY (ColFK) REFERENCES TableParente(ColPK);`
5. Supprimer une contrainte : `ALTER TABLE NomTable DROP CONSTRAINT NomContrainte;` (MySQL : `DROP FOREIGN KEY NomContrainte;`).
6. Supprimer la table : `DROP TABLE NomTable;` (structure + données + index + contraintes). `DROP TABLE IF EXISTS NomTable;` évite l'erreur si absente.

Attention : Toute commande DDL valide une transaction implicite (COMMIT) dans la plupart des SGBD.

EXERCICE RÉSOLU 9 Évolution du schéma : ajout colonne et FK

Énoncé : Table `ETUDIANT` existante. Opérations :

1. Ajouter `Telephone VARCHAR(15)` (optionnel).
2. Rendre `Email` obligatoire (`NOT NULL`).
3. Ajouter `IdTuteur INT + FK FK_Etudiant_Tuteur vers TUTEUR(IdTuteur)`.
4. Supprimer table `ANCIENS_NOTES`.

SOLUTION

Solution détaillée :

1. `ALTER TABLE ETUDIANT ADD Telephone VARCHAR(15);`
2. **MySQL** : `ALTER TABLE ETUDIANT MODIFY Email VARCHAR(100) NOT NULL;`
Standard/Access : `ALTER TABLE ETUDIANT ALTER COLUMN Email SET NOT NULL;`
3. **MySQL** (une commande) :

```
ALTER TABLE ETUDIANT  
  ADD IdTuteur INT,  
  ADD CONSTRAINT FK_Etudiant_Tuteur FOREIGN KEY (IdTuteur) REFERENCES TUTEUR(IdTuteur);
```

Séparément (portable) :

```
ALTER TABLE ETUDIANT ADD IdTuteur INT;  
ALTER TABLE ETUDIANT ADD CONSTRAINT FK_Etudiant_Tuteur FOREIGN KEY (IdTuteur) REFERENCES  
  TUTEUR(IdTuteur);
```

4. DROP TABLE IF EXISTS ANCIENS_NOTES;

Conclusion : ALTER TABLE fait évoluer sans perte (sauf DROP COLUMN), DROP TABLE supprime définitivement.

10.4 Insertion, modification et suppression des données

MÉTHODE Insérer des données (INSERT INTO)

Objectif : Ajouter une ou plusieurs lignes.

1. **Syntaxe explicite (recommandée) :** INSERT INTO NomTable (Col1, Col2, ...) VALUES (Val1, Val2, ...); **Ordre colonnes = ordre valeurs. Colonnes omises = DEFAULT ou NULL.**
2. **Syntaxe implicite (risquée) :** INSERT INTO NomTable VALUES (Val1, Val2, ...); **Doit fournir toutes les colonnes dans l'ordre exact de la définition.**
3. **Insertion multiple (MySQL, PG, SQL Server) :** INSERT INTO NomTable (Col1, Col2) VALUES (V1, V2), (V3, V4), ...; **Performant.**
4. **Littéraux :** Chaînes/Date → 'valeur' (guillemets simples). Nombres → 123. NULL → mot-clé sans quotes. Date → 'AAAA-MM-JJ'.
5. **Respect contraintes :** Échec si violation PK (doublon), NN (manquant), UQ (doublon), FK (parent inexistant).

EXERCICE RÉSOLU 10 Insertion d'élèves dans ETUDIANT

Énoncé : Insérer 3 élèves (structure exercices 1+2 : Telephone, IdTuteur ajoutés). Classes (1,2) et Tuteurs (10,20) existent.

Matricule	Nom	Prénom	DateNaiss	Sexe	Email	IdClasse	IdTuteur
E20240001	MBARGA	Jean	2006-05-12	M	jean.mbarga@email.cm	1	10
E20240002	ETOUNDI	Marie	2007-11-23	F	marie.etoundi@email.cm	1	10
E20240003	FOUDA	Paul	2005-02-28	M	paul.fouda@email.cm	2	20

SOLUTION

Solution : Syntaxe explicite multi-lignes. Telephone omis (→ NULL).

```
INSERT INTO ETUDIANT (Matricule, Nom, Prenom, DateNaiss, Sexe, Email, IdClasse, IdTuteur)
VALUES
('E20240001', 'MBARGA', 'Jean', '2006-05-12', 'M', 'jean.mbarga@email.cm', 1, 10),
('E20240002', 'ETOUNDI', 'Marie', '2007-11-23', 'F', 'marie.etoundi@email.cm', 1, 10),
('E20240003', 'FOUDA', 'Paul', '2005-02-28', 'M', 'paul.fouda@email.cm', 2, 20);
```

Vérifications : Types respectés, PK uniques, FK existantes, Email uniques, NN fournis. Atomicité : tout ou rien.

MÉTHODE**Modifier et supprimer des données (UPDATE, DELETE)**

Objectif : Changer valeurs (UPDATE) ou retirer lignes (DELETE).

1. UPDATE :

```
UPDATE NomTable
SET Col1 = NouvVal1, Col2 = NouvVal2, ...
WHERE Condition; -- INDISPENSABLE ! Sans WHERE : TOUTES lignes modifiées.
```

2. DELETE :

```
DELETE FROM NomTable
WHERE Condition; -- INDISPENSABLE ! Sans WHERE : TOUTES lignes supprimées.
```

3. Clause WHERE : Opérateurs =, <> (≠), <, >, <=, >=, AND, OR, NOT, IN, BETWEEN, LIKE, IS NULL. Parenthèses pour priorité.

4. Sous-requêtes dans WHERE : Ex : WHERE IdClasse IN (SELECT IdClasse FROM CLASSE WHERE Niveau = 'Terminale').

5. Contraintes : UPDATE échoue si violation PK, UQ, FK, NN. DELETE échoue si lignes filles référencent (sauf ON DELETE CASCADE).

Règle d'or : Tester condition avec SELECT * FROM NomTable WHERE Condition; avant UPDATE/DELETE.

EXERCICE RÉSOLU 11 Mise à jour et suppression ciblées

Énoncé : Table ETUDIANT peuplée.

1. Élève E20240001 : téléphone '6XXYYZZWW'.
2. Élèves classe IdClasse=1 → classe IdClasse=3.
3. Supprimer élève E20240003.

SOLUTION

Solution :

1. Ciblage PK (sûr) :

```
UPDATE ETUDIANT  
SET Telephone = '6XXYYZZWW'  
WHERE Matricule = 'E20240001';
```

2. Ciblage groupe (FK) :

```
UPDATE ETUDIANT  
SET IdClasse = 3  
WHERE IdClasse = 1;
```

Test préalable : `SELECT * FROM ETUDIANT WHERE IdClasse = 1;` (retourne Jean, Marie → 2 lignes modifiées).

3. Suppression PK :

```
DELETE FROM ETUDIANT  
WHERE Matricule = 'E20240003';
```

Note : Échec si FK référençant cet élève avec `ON DELETE RESTRICT` (défaut). On suppose pas de table fille ou `CASCADE/SET NULL`.

Conclusion : PK dans `WHERE` pour précision unitaire ; FK ou critère métier pour groupe.

10.5 Interrogation d'une table : SELECT

MÉTHODE SELECT simple : filtres, tri, distinct, calculs, alias

Objectif : Extraire données d'une table (projection, sélection, tri).

1. Syntaxe :

```
SELECT [DISTINCT] Col1, Col2, ... | *  
FROM NomTable  
[WHERE Condition]  
[ORDER BY Col1 [ASC|DESC], Col2 ...];
```

- 2.
- 3.
- 4.
- 5.
- 6.

EXERCICE RÉSOLU 12**Requêtes de sélection variées sur ETUDIANT**

Énoncé : Écrire les requêtes pour :

1. Nom, Prénom, Email de tous élèves, triés Nom puis Prénom.
2. Matricule, Nom des élèves nés après 2006-01-01 (strict).
3. Noms distincts des classes ayant au moins un élève (via IdClasse).
4. Nom, Prénom, Âge approx (année courante - année naissance), tri âge décroissant.

SOLUTION

Solution :

```
SELECT Nom, Prénom, Email
FROM ETUDIANT
ORDER BY Nom ASC, Prénom ASC;
```

```
SELECT Matricule, Nom
FROM ETUDIANT
WHERE DateNaiss > '2006-01-01';
```

Note : '>' exclut le 2006-01-01. Inclusif : >=.

```
SELECT DISTINCT IdClasse
FROM ETUDIANT;
```

(Pour libellé classe : jointure nécessaire, voir §6).

```
SELECT Nom, Prénom,
       (YEAR(CURDATE()) - YEAR(DateNaiss)) AS Age
FROM ETUDIANT
```

```
| ORDER BY Age DESC;
```

Précision : Âge "année civile". Âge exact MySQL : `TIMESTAMPDIFF(YEAR, DateNaiss, CURDATE())`.

Conclusion : `SELECT` ne modifie pas. `DISTINCT` sur combinaison colonnes sélectionnées. Alias lisibilité + tri sur calculé.

10.6 Jointure entre deux tables et mise en œuvre pratique

MÉTHODE Jointure interne (équi-jointure) : syntaxe programme officiel

Objectif : Combiner lignes de deux tables liées par FK (table Enfant → table Parente).

1. Identifier relation : `ETUDIANT.IdClasse (FK) → CLASSE.IdClasse (PK)`.
2. Syntaxe "ancienne" (exigible au Bac) :

```
SELECT t1.ColA, t2.ColB, ...  
FROM Table1 t1, Table2 t2  
WHERE t1.ColFK = t2.ColPK  
[AND AutresConditions];
```

3. Alias (t1, t2) : Obligatoires si noms colonnes identiques (IdClasse). Préfixer toutes colonnes (t1.Nom, t2.Libelle) pour lever ambiguïté. Recommandés toujours.
4. Condition jointure : `t1.FK = t2.PK`. `AND` implicite avec autres filtres `WHERE`.
5. Jointure externe (`LEFT/RIGHT JOIN`) : Non au programme. Sert à afficher "parents sans enfants" (ex : classes vides).

EXERCICE RÉSOLU 13 Jointure ETUDIANT – CLASSE pour bulletin

Énoncé : Tables :

- `ETUDIANT(Matricule, Nom, Prenom, IdClasse, ...)`
- `CLASSE(IdClasse, Libelle, Niveau, Salle)`

Afficher pour chaque élève : Matricule, Nom, Prenom, Libelle, Niveau, triés Niveau, Libelle, Nom.

SOLUTION

Solution :

1. Analyse : Lien `e.IdClasse = c.IdClasse`. Colonne commune → alias indispensables.

2. Alias : e (ETUDIANT), c (CLASSE).

3. Requête :

```
SELECT e.Matricule, e.Nom, e.Prenom, c.Libelle, c.Niveau
FROM ETUDIANT e, CLASSE c
WHERE e.IdClasse = c.IdClasse
ORDER BY c.Niveau, c.Libelle, e.Nom;
```

4. Explication :

- (a) FROM ETUDIANT e, CLASSE c : **Produit cartésien.**
- (b) WHERE e.IdClasse = c.IdClasse : **Filtre équi-jointure interne. Exclut élèves sans classe (NULL) et classes vides.**
- (c) SELECT ... : **Projection préfixée.**
- (d) ORDER BY ... : **Tri classes puis élèves.**

Conclusion : Jointure interne = correspondances exactes. Alias courts évitent erreur "Column 'IdClasse' in where clause is ambiguous".

MÉTHODE**Création concrète : Console MySQL / Interface Access**

Objectif : Réaliser la création physiquement.

1. MySQL (Terminal) :

- (a) Connexion : `mysql -u root -p` (mot de passe).
- (b) Créer base : `CREATE DATABASE NomBase CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;` (UTF-8 complet).
- (c) Sélectionner : `USE NomBase;`.
- (d) Exécuter `CREATE TABLE ... ;` (copier-coller ou SOURCE fichier.sql);.
- (e) Vérifier : `SHOW TABLES;`, `DESCRIBE NomTable;`.

2. Access (Graphique) :

- (a) Fichier → Nouveau → Base vide → Nommer (GestionLycee.accdb) → Créer.
- (b) Onglet Créer → Conception Table.
- (c) Saisir : Nom champ, Type (Texte Court ≈ VARCHAR, Numérique ≈ INT, Date/Heure ≈ DATE). Sélectionner PK → bouton Clé primaire.
- (d) Propriétés (bas) : Requis = Oui (NOT NULL), Indexé = Oui (Sans doublons) (UNIQUE/PK).
- (e) Enregistrer (Ctrl+S) → Nom table (T_Etudiant).

(f) Relations : Outils → Relations → Ajouter tables → Glisser PK vers FK → Cocher Intégrité référentielle.

EXERCICE RÉSOLU 14 Script complet MySQL pour base "LyceeDB"

Énoncé : Script `creation.sql` pour :

1. Base LyceeDB UTF-8.
2. Table CLASSE(IdClasse INT PK, Libelle VARCHAR(30) NN, Niveau VARCHAR(20)).
3. Table ETUDIANT (structure exo 1, FK vers CLASSE).
4. Insérer 2 classes : (1, 'Terminale C1', 'Terminale'), (2, 'Terminale D1', 'Terminale').
5. Insérer élève ('E20240001', 'MBARGA', 'Jean', '2006-05-12', 'M', 'jean.mbarga@email.cm', 1).

SOLUTION

Solution :

```
-- 1. Création base de données
CREATE DATABASE IF NOT EXISTS LyceeDB
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_general_ci;

-- 2. Utilisation base
USE LyceeDB;

-- 3. Table CLASSE (parente, créée EN PREMIER)
CREATE TABLE CLASSE (
  IdClasse INT PRIMARY KEY,
  Libelle VARCHAR(30) NOT NULL,
  Niveau VARCHAR(20)
);

-- 4. Table ETUDIANT (enfant, FK vers CLASSE)
CREATE TABLE ETUDIANT (
  Matricule CHAR(10) PRIMARY KEY,
  Nom VARCHAR(50) NOT NULL,
  Prenom VARCHAR(50) NOT NULL,
  DateNaiss DATE NOT NULL,
  Sexe CHAR(1) NOT NULL,
  Email VARCHAR(100) UNIQUE,
  IdClasse INT NOT NULL,
  CONSTRAINT FK_Etudiant_Classe
    FOREIGN KEY (IdClasse) REFERENCES CLASSE(IdClasse)
);

-- 5. Données de référence (classes)
INSERT INTO CLASSE (IdClasse, Libelle, Niveau) VALUES
(1, 'Terminale C1', 'Terminale'),
```

```
(2, 'Terminale D1', 'Terminale');  
  
-- 6. Données transactionnelles (élève)  
INSERT INTO ETUDIANT (Matricule, Nom, Prenom, DateNaiss, Sexe, Email, IdClasse)  
VALUES ('E20240001', 'MBARGA', 'Jean', '2006-05-12', 'M', 'jean.mbarga@email.cm', 1);
```

Exécution console :

```
mysql -u root -p < creation.sql  
# ou dans client mysql :  
SOURCE /chemin/vers/creation.sql;
```

Conclusion : Ordre crucial : Base → Tables parentes (sans FK) → Tables enfants (avec FK) → Données référence → Données transactionnelles. utf8mb4 gère accents, alphabets locaux, émojis.

ATTENTION Les 5 erreurs qui coûtent des points au Bac

1. Oublier `WHERE` dans `UPDATE` ou `DELETE`. → Modification/suppression de toutes lignes. Réflexe : Écrire `WHERE` d'abord, tester avec `SELECT`.
2. Ambiguïté noms colonnes en jointure. `WHERE IdClasse = IdClasse` au lieu de `WHERE e.IdClasse = c.IdClasse`. → Erreur "Column 'IdClasse' in where clause is ambiguous". Réflexe : Alias courts (`e`, `c`) + préfixer toutes colonnes (`e.Nom`, `c.Libelle`).
3. Mauvais format date/chaîne. `DateNaiss = 2006-05-12` (soustraction!) ou `'12/05/2006'` (format FR). → Erreur/résultat faux. Règle : ISO `'AAAA-MM-JJ'` entre quotes simples.
4. Ordre création tables (FK). Créer enfant (`ETUDIANT`) avant parente (`CLASSE`). → Erreur "Referenced table 'CLASSE' not found". Règle : Tables référencées d'abord.
5. Confondre `DROP TABLE` et `DELETE FROM`. `DROP TABLE` supprime la structure (table disparaît). `DELETE FROM` supprime les données (table reste vide). "Vider la table" → `DELETE FROM` (ou `TRUNCATE TABLE`). "Supprimer la table" → `DROP TABLE`.

LESAVAIS-TU ? Le Cameroun et les SGBD libres

Au Cameroun, l'administration publique (MINPOSTEL, MINESEC, MINSANTE) migre progressivement vers des solutions open source (PostgreSQL, MySQL/MariaDB) pour la souveraineté numérique et la réduction des coûts de licences. La maîtrise de SQL standard et de MySQL en console est donc une compétence stratégique pour l'insertion professionnelle (développeur, administrateur système, data analyst). Le projet CAMTEL Fibre Optique Nationale interconnecte les data centers hébergeant ces SGBD.

11 EXERCICES

11.1 Je vérifie mes connaissances

EXERCICE 8 QCM : SGBD et langage SQL

Parmi les affirmations suivantes, une seule est exacte. Laquelle ?

1. MySQL est un langage de programmation orienté objet.
2. Le langage SQL permet uniquement d'interroger les données (SELECT).
3. Un SGBD (Système de Gestion de Base de Données) assure l'interface entre l'utilisateur et la base de données physique.
4. La commande `CREATE DATABASE` sert à insérer des lignes dans une table.

EXERCICE 9 Vrai ou Faux justifié

Dire si les affirmations suivantes sont vraies ou fausses. Justifiez brièvement chaque réponse.

1. Le type de données `VARCHAR(50)` occupe toujours 50 octets en mémoire pour chaque valeur stockée.
2. La contrainte `PRIMARY KEY` implique automatiquement `NOT NULL` et l'unicité des valeurs.
3. La commande `DROP TABLE` supprime la structure de la table mais conserve les données dans un fichier de sauvegarde.
4. Dans une requête `SELECT`, l'utilisation de `DISTINCT` élimine les lignes dupliquées dans le résultat.

EXERCICE 10 Définitions et correspondance

Associez chaque concept de la colonne de gauche à sa définition exacte dans la colonne de droite.

Concept	Définition
SGBD	A. Langage standardisé pour interroger et manipuler une base de données relationnelle.
SQL	B. Logiciel permettant de créer, gérer et exploiter une base de données.
Clé primaire	C. Attribut (ou groupe d'attributs) identifiant de manière unique chaque occurrence d'une entité.
Jointure	D. Opération relationnelle combinant les lignes de deux tables grâce à une condition de liaison.

EXERCICE 11 Calcul direct : cardinalité d'un résultat

Soit la table `PRODUIT` suivante :

CodeP	Designation	Categorie	Prix
P1	Stylo Bille	Papeterie	200
P2	Cahier 100p	Papeterie	500
P3	Clé USB 16Go	Informatique	3500
P4	Souris Optique	Informatique	2500
P5	Ramette A4	Papeterie	2500

Combien de lignes renverra la requête suivante ?

```
SELECT DISTINCT Categorie FROM PRODUIT WHERE Prix >= 2500 ;
```

11.2 Je m'entraîne

EXERCICE 12 Création de table avec contraintes

On souhaite créer une table `ETUDIANT` pour gérer les inscriptions dans un établissement secondaire. La structure est la suivante :

- `Matricule` : chaîne de 10 caractères, clé primaire.
- `Nom` : chaîne de 30 caractères, obligatoire.
- `Prenom` : chaîne de 30 caractères.
- `DateNaiss` : date, obligatoire.
- `Sexe` : un seul caractère ('M' ou 'F').
- `Classe` : chaîne de 15 caractères (ex : 'Terminale C').

Écrire la requête SQL `CREATE TABLE` correspondante, en intégrant les contraintes `PRIMARY KEY` et `NOT NULL` appropriées.

EXERCICE 13 Manipulation de données : INSERT, UPDATE, DELETE

La table `EMPLOYE` existe avec la structure : `Matricule (PK, INT)`, `Nom (VARCHAR)`, `Service (VARCHAR)`, `Salaire (INT)`, `DateEmbauche (DATE)`. Écrire les requêtes SQL pour effectuer les opérations suivantes :

1. Insérer un nouvel employé : `Matricule 105`, `Nom 'MBARGA'`, `Service 'Informatique'`, `Salaire 250000`, `DateEmbauche '2023-09-01'`.
2. Augmenter de 10% le salaire de tous les employés du service `'Comptabilité'`.
3. Supprimer les employés embauchés avant le 1er janvier 1990.

EXERCICE 14 Interrogation simple et opérateurs logiques

On considère la table `LIVRE` (`ISBN PK`, `Titre`, `Auteur`, `AnneeEdition`, `Prix`, `Stock`). Écrire les requêtes `SELECT` pour :

1. Afficher le titre et l'auteur des livres publiés après 2010.
2. Afficher toutes les colonnes des livres dont le prix est compris entre 3000 et 10000 FCFA (bornes incluses) ET dont le stock est strictement supérieur à 5.
3. Afficher les titres distincts des livres dont l'auteur est `'BETI'` OU `'OYONO'`.
4. Compter le nombre de livres en stock (`stock > 0`) par auteur (utiliser `GROUP BY`).

EXERCICE 15 Jointure entre deux tables

On dispose de deux tables :

- `CLIENT (NumCli PK, NomCli, Ville)`
- `COMMANDE (NumCom PK, DateCom, Montant, NumCli FK référençant CLIENT)`

Écrire la requête SQL (avec alias) qui affiche, pour chaque commande, le numéro de commande, la date, le montant et le nom du client correspondant. Expliquer en une phrase le rôle de la condition de jointure dans la clause `WHERE`.

11.3 Je me prépare au Bac

EXERCICE 16 **Sujet Bac Type : Gestion d'une bibliothèque scolaire – Partie 1 : Modélisation et Création**

Le lycée bilingue de Ngaoundéré souhaite informatiser la gestion de sa bibliothèque. On s'intéresse aux entités `LIVRE` et `ELEVE` et à l'association `EMPRUNTER`.

- Un livre est identifié par son `ISBN` (chaîne 13), a un `Titre` (chaîne 100, non null), un `Auteur` (chaîne 50), une `Categorie` (chaîne 30) et un `NbExemplaires` (entier).
 - Un élève est identifié par son `Matricule` (entier), a un `Nom` (chaîne 30, non null), un `Prenom` (chaîne 30), une `Classe` (chaîne 20) et une `DateNaissance` (date).
 - Un élève peut emprunter plusieurs livres. Un exemplaire de livre peut être emprunté par un seul élève à la fois. On note la `DateEmprunt` (date) et la `DateRetourPrevue` (date).
1. Proposer le MCD (Modèle Conceptuel de Données) sous forme textuelle : Entités, Attributs (clé primaire soulignée), Associations avec cardinalités.
 2. Traduire ce MCD en MLD (Modèle Logique de Données) : donner le schéma relationnel normalisé (tables, clés primaires, clés étrangères).
 3. Écrire la requête SQL `CREATE TABLE` pour la table `LIVRE` avec toutes les contraintes d'intégrité (clé primaire, non null).
 4. Écrire la requête SQL `CREATE TABLE` pour la table `EMPRUNT` (table de liaison) avec ses clés primaires composées et ses clés étrangères référençant `LIVRE` et `ELEVE`.

EXERCICE 17 **Sujet Bac Type : Gestion du personnel – Requêtes complexes**

La base de données `PERSONNEL` d'une entreprise camerounaise contient deux tables :

EMPLOYE	SERVICE
<u>Matricule</u> (INT)	<u>CodeServ</u> (CHAR(3))
Nom (VARCHAR(30))	NomServ (VARCHAR(30))
Prenom (VARCHAR(30))	Localisation (VARCHAR(30))
Salaire (INT)	ChefServ (INT) – Matricule du chef
DateEmb (DATE)	
CodeServ (CHAR(3)) – Clé étrangère	

Données d'exemple pour `EMPLOYE` :

Matricule	Nom	Prenom	Salaire	DateEmb	CodeServ
101	FOPA	Jean	350000	2015-01-10	INF
102	NGONO	Marie	280000	2018-05-20	COM
103	KAMGA	Paul	420000	2010-11-01	INF
104	ETOUNDI	Alice	310000	2020-03-15	RH
105	MEKONGO	Pierre	290000	2019-09-01	COM

Données d'exemple pour SERVICE :

CodeServ	NomServ	Localisation	ChefServ
INF	Informatique	Batiment A	103
COM	Comptabilité	Batiment B	102
RH	Ressources Humaines	Batiment A	104

Écrire les requêtes SQL pour :

1. Afficher le nom, le prénom et le salaire des employés gagnant plus que la moyenne des salaires de l'entreprise.
2. Afficher le nom de chaque service et le nombre d'employés qui y travaillent (y compris les services vides).
3. Augmenter de 5% le salaire des employés du service 'Informatique' embauchés avant 2016.
4. Supprimer l'employé 'MEKONGO Pierre'. Que se passe-t-il s'il est chef d'un service ? (Expliquer sans écrire de code).
5. Afficher pour chaque employé : son nom, son prénom, le nom de son service et le nom du chef de ce service (auto-jointure ou double jointure nécessaire).

EXERCICE 18 Sujet Bac Type : Débogage et Mise en œuvre pratique

Un élève de Terminale a écrit le script SQL ci-dessous pour créer la base de données d'un cybercafé gérant les CLIENTS et leurs SESSIONS. Ce script contient 5 erreurs (syntaxe, logique, ordre d'exécution, types, contraintes).

```
-- Script élève
CREATE DATABASE CYBERCAFE;
USE CYBERCAFE;

CREATE TABLE SESSION (
  IdSession INT PRIMARY KEY,
  DateDebut DATE,
  Duree INT, -- en minutes
  IdClient INT,
```

```
FOREIGN KEY (IdClient) REFERENCES CLIENT(IdClient)
);

CREATE TABLE CLIENT (
  IdClient INT PRIMARY KEY,
  Nom VARCHAR(30) NOT NULL,
  Prenom VARCHAR(30),
  Solde INT DEFAULT '0.00', -- Erreur type
  DateInscription DATE DEFAULT CURRENT_DATE
);

INSERT INTO CLIENT VALUES (1, 'ABENA', 'Paul', 5000, '15/01/2024'); -- Erreur format date
INSERT INTO SESSION VALUES (101, '2024-01-15', 60, 1);
INSERT INTO SESSION VALUES (102, '2024-01-15', 45, 2); -- Client 2 inexistant

UPDATE CLIENT SET Solde = Solde - 1000; -- Erreur logique

SELECT C.Nom, S.DateDebut, S.Duree
FROM CLIENT C, SESSION S; -- Erreur jointure
```

1. Identifier les 5 erreurs présentes dans ce script en précisant le numéro de ligne (ou le bloc concerné) et la nature de l'erreur.
2. Proposer le script corrigé, complet et ordonnancé correctement.
3. Écrire la requête SQL permettant d'afficher le nom des clients et la durée totale de leurs sessions pour le mois de janvier 2024 (utiliser SUM et GROUP BY).

12 CORRIGÉS DES EXERCICES

CORRIGÉ Exercice 1 — QCM : SGBD et langage SQL

Réponse exacte : c.

Un **SGBD** (Système de Gestion de Base de Données) est le logiciel qui assure l'interface entre l'utilisateur (ou les applications) et la base de données physique : il gère le stockage, la sécurité, les accès concurrents et l'intégrité des données. Access et MySQL en sont des exemples.

Pourquoi les autres propositions sont fausses :

- a. Faux. MySQL est un **SGBD** relationnel, pas un langage de programmation. Le langage qu'il utilise pour dialoguer avec la base est le SQL.
- b. Faux. SQL ne sert pas qu'à interroger : il comprend le LDD (CREATE, ALTER, DROP), le LMD (INSERT, UPDATE, DELETE, SELECT) et le LCD (Langage de Contrôle des Données / DCL : droits d'accès GRANT, REVOKE).
- d. Faux. CREATE DATABASE crée une base de données vide. L'insertion de lignes se fait avec INSERT INTO.

CORRIGÉ Exercice 2 — Vrai ou Faux justifié

1. Faux. VARCHAR(50) est une chaîne de longueur variable : seuls les caractères réellement stockés occupent de la place (plus un ou deux octets pour la longueur). La valeur 'ABENA' n'occupe donc que 5 caractères, pas 50. C'est le type CHAR(50) qui occupe toujours 50 caractères (complétés par des espaces).
2. Vrai. Une clé primaire identifie de manière unique chaque ligne : ses valeurs doivent donc être uniques et ne peuvent pas être nulles. Déclarer PRIMARY KEY implique automatiquement NOT NULL et UNIQUE.
3. Faux. DROP TABLE supprime définitivement la structure et toutes les données de la table. Aucune sauvegarde automatique n'est créée : c'est une commande irréversible, à manipuler avec la plus grande prudence.
4. Vrai. Le paramètre DISTINCT, placé juste après SELECT, élimine les doublons : chaque combinaison de valeurs des colonnes affichées n'apparaît qu'une seule fois dans le résultat.

CORRIGÉ Exercice 3 — Définitions et correspondance

Les bonnes associations sont :

Concept	Définition correcte
SGBD	B. Logiciel permettant de créer, gérer et exploiter une base de données.
SQL	A. Langage standardisé pour interroger et manipuler une base de données relationnelle.
Clé primaire	C. Attribut (ou groupe d'attributs) identifiant de manière unique chaque occurrence d'une entité.
Jointure	D. Opération relationnelle combinant les lignes de deux tables grâce à une condition de liaison.

Point de vigilance : ne pas confondre le SGBD (le logiciel : Access, MySQL) avec le SQL (le langage utilisé pour communiquer avec ce logiciel).

CORRIGÉ Exercice 4 — Calcul direct : cardinalité d'un résultat

La requête s'exécute en deux étapes logiques.

Étape 1 : filtrage par `WHERE Prix >= 2500`.

CodeP	Designation	Categorie	Prix
P3	Clé USB 16Go	Informatique	3500
P4	Souris Optique	Informatique	2500
P5	Ramette A4	Papeterie	2500

Les lignes P1 (200) et P2 (500) sont éliminées. Attention : la condition est `>= 2500`, donc P4 et P5 (prix exactement 2500) sont retenues.

Étape 2 : projection sur `Categorie` avec `DISTINCT`.

Les catégories des lignes retenues sont : Informatique, Informatique, Papeterie. Après élimination des doublons, il reste : Informatique, Papeterie.

La requête renvoie donc 2 lignes.

Point de vigilance : l'ordre logique d'exécution est `FROM` → `WHERE` → `SELECT DISTINCT`. Le `DISTINCT` s'applique après le filtrage, sur les catégories des lignes déjà sélectionnées.

CORRIGÉ Exercice 5 — Création de table avec contraintes

```
CREATE TABLE ETUDIANT (
  Matricule CHAR(10) PRIMARY KEY,
  Nom VARCHAR(30) NOT NULL,
  Prenom VARCHAR(30),
  DateNaiss DATE NOT NULL,
  Sexe CHAR(1),
  Classe VARCHAR(15)
```

```
);
```

Justifications des choix :

- **Matricule** est déclaré `CHAR(10)` car sa longueur est fixe (10 caractères) ; il est `PRIMARY KEY`, ce qui garantit unicité et non-nullité.
- **Nom** et **DateNaiss** portent la contrainte `NOT NULL` car l'énoncé les qualifie d'obligatoires.
- **Sexe** est un `CHAR(1)` : un seul caractère ('M' ou 'F'). Un `VARCHAR(1)` serait inutilement complexe.
- **Prenom** et **Classe** restent sans contrainte : ils peuvent être laissés vides (`NULL`).

Points de vigilance : ne pas oublier la virgule entre chaque définition de colonne (mais jamais après la dernière) ; les noms de colonnes ne prennent pas d'accents ni d'espaces.

CORRIGÉ Exercice 6 — Manipulation de données : INSERT, UPDATE, DELETE

1. Insertion du nouvel employé :

```
INSERT INTO EMPLOYE (Matricule, Nom, Service, Salaire, DateEmbauche)
VALUES (105, 'MBARGA', 'Informatique', 250000, '2023-09-01');
```

Les chaînes de caractères et les dates sont encadrées par des apostrophes ; le format de date standard SQL est 'AAAA-MM-JJ'.

2. Augmentation de 10 % des salaires du service Comptabilité :

```
UPDATE EMPLOYE
SET Salaire = Salaire * 1.10
WHERE Service = 'Comptabilite';
```

Augmenter de 10 % revient à multiplier par $1 + \frac{10}{100} = 1,10$. (On écrit 1.10 dans le code, 1,10 en français).

3. Suppression des employés embauchés avant 1990 :

```
DELETE FROM EMPLOYE
WHERE DateEmbauche < '1990-01-01';
```

Points de vigilance :

- Sans clause `WHERE`, `UPDATE` et `DELETE` s'appliquent à toutes les lignes de la table : c'est l'erreur la plus grave en manipulation de données.
- Les comparaisons de dates fonctionnent directement avec le format 'AAAA-MM-JJ' car l'ordre alphabétique y coïncide avec l'ordre chronologique.

CORRIGÉ

Exercice 7 — Interrogation simple et opérateurs logiques

1. Titre et auteur des livres publiés après 2010 :

```
SELECT Titre, Auteur  
FROM LIVRE  
WHERE AnneeEdition > 2010 ;
```

« Après 2010 » signifie strictement supérieur : un livre de 2010 est exclu.

2. Toutes les colonnes, prix entre 3000 et 10000 inclus ET stock > 5 :

```
SELECT *  
FROM LIVRE  
WHERE Prix >= 3000 AND Prix <= 10000 AND Stock > 5 ;
```

L'étoile * sélectionne toutes les colonnes. Les bornes incluses imposent >= et <= ; le stock « strictement supérieur à 5 » impose >.

3. Titres distincts des livres de BETI ou OYONO :

```
SELECT DISTINCT Titre  
FROM LIVRE  
WHERE Auteur = 'BETI' OR Auteur = 'OYONO' ;
```

DISTINCT évite d'afficher deux fois le même titre si deux éditions existent.

4. Nombre de livres en stock par auteur :

```
SELECT Auteur, COUNT(*) AS NombreLivres  
FROM LIVRE  
WHERE Stock > 0  
GROUP BY Auteur ;
```

Le WHERE filtre les lignes avant le regroupement ; GROUP BY forme ensuite un groupe par auteur et COUNT(*) compte les lignes de chaque groupe.

Point de vigilance : ne pas écrire `Auteur = 'BETI' OR 'OYONO'` : chaque condition après OR doit être complète.

CORRIGÉ

Exercice 8 — Jointure entre deux tables

```
SELECT CO.NumCom, CO.DateCom, CO.Montant, CL.NomCli  
FROM COMMANDE CO, CLIENT CL  
WHERE CO.NumCli = CL.NumCli ;
```

Rôle de la condition de jointure : la condition `CO.NumCli = CL.NumCli` associe à chaque commande le client qui l'a passée, en évitant le produit cartésien (qui apparierait chaque commande avec tous les clients, y compris les mauvais).

Points de vigilance :

- Les alias CO et CL, déclarés dans le FROM, allègent l'écriture et lèvent l'ambiguïté sur la colonne NumCli présente dans les deux tables.

- Sans la condition de jointure, le résultat contiendrait $n \times m$ lignes (produit cartésien), ce qui est presque toujours une erreur.

CORRIGÉ Exercice 9 — Sujet Bac : bibliothèque scolaire, modélisation et création

1. MCD sous forme textuelle :

- LIVRE (ISBN, Titre, Auteur, Categorie, NbExemplaires)
- ELEVE (Matricule, Nom, Prenom, Classe, DateNaissance)
- EMPRUNTER : association entre ELEVE et LIVRE, porteuse des attributs DateEmprunt et DateRetourPrevue. Cardinalités : un élève peut emprunter plusieurs livres et un livre (titre) peut être emprunté plusieurs fois (au cours du temps), donc ELEVE (0,n) — EM-PRUNTER — (0,n) LIVRE.

L'énoncé « un exemplaire ne peut être emprunté que par un seul élève à la fois » ne change pas la cardinalité (0,n) côté livre : sur toute la durée de vie de la bibliothèque, un même titre connaîtra plusieurs emprunts successifs.

2. MLD (schéma relationnel) :

- LIVRE(ISBN, Titre, Auteur, Categorie, NbExemplaires)
- ELEVE(Matricule, Nom, Prenom, Classe, DateNaissance)
- EMPRUNT(ISBN#, Matricule#, DateEmprunt, DateRetourPrevue) où ISBN référence LIVRE(ISBN) et Matricule référence ELEVE(Matricule).

L'association (0,n)–(0,n) devient une table de liaison dont la clé primaire est composée des deux clés étrangères.

3. Création de la table LIVRE :

```
CREATE TABLE LIVRE (
  ISBN      CHAR(13)  PRIMARY KEY,
  Titre     VARCHAR(100) NOT NULL,
  Auteur    VARCHAR(50),
  Categorie VARCHAR(30),
  NbExemplaires INT
);
```

4. Création de la table EMPRUNT :

```
CREATE TABLE EMPRUNT (
  ISBN      CHAR(13),
  Matricule  INT,
  DateEmprunt DATE NOT NULL,
  DateRetourPrevue DATE,
  PRIMARY KEY (ISBN, Matricule),
  FOREIGN KEY (ISBN) REFERENCES LIVRE(ISBN),
  FOREIGN KEY (Matricule) REFERENCES ELEVE(Matricule)
);
```

Barème indicatif (sur 10 points) : MCD correct (entités, attributs, clés soulignées, cardi-

nalités) : 3 pts ; MLD avec clés primaires et étrangères bien repérées : 3 pts ; CREATE TABLE LIVRE : 2 pts ; CREATE TABLE EMPRUNT avec clé primaire composée et deux REFERENCES : 2 pts.

Points de vigilance : la table EMPRUNT doit être créée après LIVRE et ELEVE (on ne peut référencer que des tables existantes) ; la clé primaire composée se déclare séparément avec PRIMARY KEY (ISBN, Matricule), pas en répétant PRIMARY KEY sur chaque colonne.

CORRIGÉ Exercice 10 — Sujet Bac : gestion du personnel, requêtes complexes

1. Employés gagnant plus que la moyenne :

```
SELECT Nom, Prenom, Salaire
FROM EMPLOYE
WHERE Salaire > (SELECT AVG(Salaire) FROM EMPLOYE);
```

La sous-requête calcule la moyenne générale : $(350000 + 280000 + 420000 + 310000 + 290000) / 5 = 1650000 / 5 = 330000$. Seuls FOPA Jean (350 000) et KAMGA Paul (420 000) sont affichés.

2. Nombre d'employés par service, services vides inclus :

```
SELECT S.NomServ, COUNT(E.Matricule) AS NbEmployes
FROM SERVICE S LEFT JOIN EMPLOYE E ON S.CodeServ = E.CodeServ
GROUP BY S.NomServ;
```

La jointure externe LEFT JOIN conserve les services sans employé (qui affichent 0 car COUNT(E.Matricule) ne compte pas les NULL). Résultat attendu : Informatique : 2 ; Comptabilité : 2 ; Ressources Humaines : 1.

3. Augmentation de 5 % ciblée :

```
UPDATE EMPLOYE
SET Salaire = Salaire * 1.05
WHERE CodeServ = 'INF' AND DateEmb < '2016-01-01';
```

Les employés du service Informatique (CodeServ = 'INF') embauchés avant le 2016-01-01 sont FOPA Jean (embauché le 2015-01-10) et KAMGA Paul (embauché le 2010-03-15). Les deux sont concernés. Nouveaux salaires : FOPA $350000 \times 1,05 = 367500$ FCFA ; KAMGA $420000 \times 1,05 = 441000$ FCFA.

4. Suppression de MEKONGO Pierre :

```
DELETE FROM EMPLOYE
WHERE Nom = 'MEKONGO' AND Prenom = 'Pierre';
```

MEKONGO (matricule 105) n'est chef d'aucun service (les chefs sont 103, 102, 104), donc la suppression s'effectue normalement. En revanche, si l'employé supprimé était référencé comme ChefServ dans la table SERVICE, le SGBD refuserait la suppression pour préserver l'intégrité référentielle : on ne peut pas supprimer une ligne tant qu'une clé étrangère d'une autre table pointe vers elle. Il faudrait d'abord modifier ou supprimer la

référence dans SERVICE.

5. Employé, son service et le nom du chef :

```
SELECT E.Nom, E.Prenom, S.NomServ, C.Nom AS NomChef
FROM EMPLOYEE E, SERVICE S, EMPLOYE C
WHERE E.CodeServ = S.CodeServ
      AND S.ChefServ = C.Matricule;
```

La table EMPLOYEE est utilisée deux fois sous deux alias différents (E pour l'employé, C pour le chef) : c'est une auto-jointure. Résultat attendu :

- FOPA / Informatique / KAMGA
- NGONO / Comptabilité / NGONO (elle est sa propre chef)
- KAMGA / Informatique / KAMGA (il est son propre chef)
- ETOUNDI / Ressources Humaines / ETOUNDI (il est son propre chef)
- MEKONGO / Comptabilité / NGONO

Barème indicatif (sur 10 points) : 2 pts par requête correcte, dont 1 pt pour la justification ou l'explication dans les questions 4 et 5.

Points de vigilance : citer explicitement l'intégrité référentielle à la question 4 ; à la question 5, exiger deux alias distincts et deux conditions de jointure dans le WHERE.

CORRIGÉ Exercice 11 — Sujet Bac : débogage et mise en œuvre pratique

1. Les 5 erreurs :

1. Ordre de création des tables (bloc CREATE TABLE SESSION) : SESSION référence CLIENT avec une clé étrangère alors que CLIENT n'existe pas encore. Il faut créer CLIENT avant SESSION.
2. Type incohérent (ligne Solde INT DEFAULT '0.00') : la valeur par défaut '0.00' est une chaîne décimale entre apostrophes, incompatible avec le type INT. Il faut écrire DEFAULT 0.
3. Format de date (premier INSERT) : '15/01/2024' n'est pas le format SQL standard ; il faut '2024-01-15'.
4. Violation d'intégrité référentielle (troisième INSERT) : la session 102 référence le client 2 qui n'existe pas dans CLIENT ; le SGBD rejette l'insertion.
5. Absence de condition de jointure (dernier SELECT) : FROM CLIENT C, SESSION S sans WHERE produit un produit cartésien ; il manque WHERE C.IdClient = S.IdClient.

(On peut aussi signaler le UPDATE sans WHERE, qui débite tous les clients : c'est une erreur logique acceptable comme 5^e réponse.)

2. Script corrigé et ordonnancé (Syntaxe MySQL) :

```
CREATE DATABASE CYBERCAFE;
```

```
USE CYBERCAFE;

-- Creation de la table parente d'abord
CREATE TABLE CLIENT (
    IdClient INT PRIMARY KEY,
    Nom VARCHAR(30) NOT NULL,
    Prenom VARCHAR(30),
    Solde INT DEFAULT 0,
    DateInscription DATE
);

-- Puis la table qui la reference
CREATE TABLE SESSION (
    IdSession INT PRIMARY KEY,
    DateDebut DATE,
    Duree INT,
    IdClient INT,
    FOREIGN KEY (IdClient) REFERENCES CLIENT(IdClient)
);

INSERT INTO CLIENT VALUES (1, 'ABENA', 'Paul', 5000, '2024-01-15');
INSERT INTO CLIENT VALUES (2, 'NDJOCK', 'Marie', 3000, '2024-01-15');
INSERT INTO SESSION VALUES (101, '2024-01-15', 60, 1);
INSERT INTO SESSION VALUES (102, '2024-01-15', 45, 2);

UPDATE CLIENT SET Solde = Solde - 1000 WHERE IdClient = 1;

SELECT C.Nom, S.DateDebut, S.Duree
FROM CLIENT C, SESSION S
WHERE C.IdClient = S.IdClient;
```

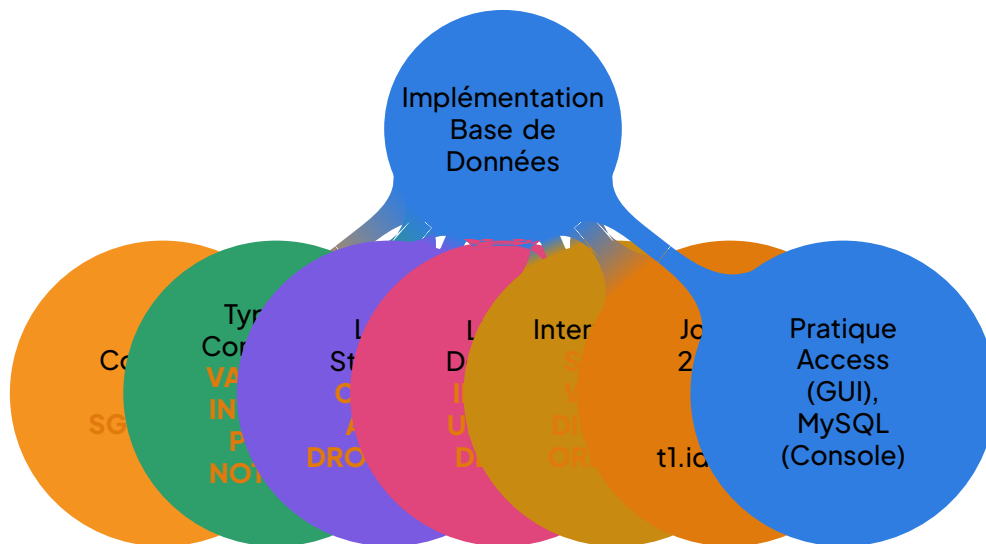
3. Durée totale des sessions par client en janvier 2024 :

```
SELECT C.Nom, SUM(S.Duree) AS DureeTotale
FROM CLIENT C, SESSION S
WHERE C.IdClient = S.IdClient
    AND S.DateDebut >= '2024-01-01'
    AND S.DateDebut <= '2024-01-31'
GROUP BY C.Nom;
```

Barème indicatif (sur 10 points) : identification des 5 erreurs : 5 pts (1 pt par erreur correctement localisée et qualifiée) ; script corrigé cohérent : 3 pts ; requête avec SUM, GROUP BY et jointure : 2 pts.

Points de vigilance : l'ordre de création des tables (parente avant fille) est une justification attendue ; le format de date 'AAAA-MM-JJ' doit être cité ; la condition de jointure dans le WHERE est indispensable dès que deux tables figurent dans le FROM.

13 FICHE BILAN



Carte mentale : Leçon 6 — Implémentation d'une base de données

FICHE BILAN L'essentiel à connaître pour l'examen

13.1 Définitions clés

DÉFINITION **Base de Données (BDD)**

Collection structurée de données persistantes, organisées pour un accès, une gestion et une mise à jour faciles. Modélisée par un **MCD** puis un **MLD**.

DÉFINITION SGBD (Système de Gestion de Base de Données)

Logiciel (ex. MySQL, Access, PostgreSQL) permettant de créer, gérer, sécuriser et interroger une BDD. Assure l'indépendance données/programmes.

DÉFINITION SQL (Structured Query Language)

Langage standardisé pour manipuler les BDD relationnelles. Deux sous-langages principaux :

- LDD (Langage de Définition des Données) : **CREATE**, **ALTER**, **DROP** (structure).
- LMD (Langage de Manipulation des Données) : **INSERT**, **UPDATE**, **DELETE**, **SELECT** (contenu).

DÉFINITION Clé Primaire (Primary Key – PK)

Attribut (ou groupe d'attributs) identifiant uniquement chaque ligne (tuple) d'une table. Non NULL, Unique.

DÉFINITION Clé Étrangère (Foreign Key – FK)

Attribut d'une table faisant référence à la **PK** d'une autre table. Assure l'intégrité référentielle.

13.2 Types de données courants (MySQL / Access)

Type SQL	Description / Usage	Exemple
INT / INTEGER	Entier (clé primaire, compteurs, quantités)	id_eleve INT
VARCHAR(n)	Chaîne variable (noms, adresses, max <i>n</i> caractères)	nom VARCHAR(50)
CHAR(n)	Chaîne fixe (codes, matricules, <i>n</i> constant)	sexe CHAR(1)
DATE	Date calendrier (naissance, commande)	dt_naiss DATE
DECIMAL(p,s) / NUMERIC	Nombre exact (prix, notes, <i>p</i> chiffres totaux, <i>s</i> décimales)	prix DECIMAL(10,2)

13.3 Contraintes d'intégrité (à poser dans CREATE TABLE)

Contrainte	Rôle & Syntaxe
PRIMARY KEY	Identifiant unique. <code>PRIMARY KEY (id)</code> ou <code>id INT PRIMARY KEY</code>
NOT NULL	Interdit la valeur absente. <code>nom VARCHAR(50) NOT NULL</code>
UNIQUE	Valeurs distinctes (autre que PK). <code>email VARCHAR(100) UNIQUE</code>
DEFAULT val	Valeur par défaut. <code>statut VARCHAR(20) DEFAULT 'Actif'</code>
REFERENCES / FOREIGN KEY	Lien vers PK table parente. <code>id_classe INT REFERENCES Classe(id)</code>

13.4 Méthodes express (Syntaxe de référence)

MÉTHODE Créer une table (LDD)

1. `CREATE TABLE NomTable (`
2. `col1 TYPE CONTRAINTES,`
3. `col2 TYPE CONTRAINTES,`
4. `PRIMARY KEY (col1), – ou inline`
5. `FOREIGN KEY (col_fk) REFERENCES AutreTable(pk)`
6. `);`

MÉTHODE Modifier une table (ALTER TABLE)

- Ajouter colonne : `ALTER TABLE T ADD colonne TYPE CONTRAINTES;`
- Supprimer colonne : `ALTER TABLE T DROP COLUMN colonne;`
- Modifier type/contrainte : `ALTER TABLE T MODIFY colonne NEW_TYPE;` (MySQL) / `ALTER TABLE T ALTER COLUMN colonne NEW_TYPE;` (Access/SQL Server)
- Ajouter clé étrangère : `ALTER TABLE T ADD FOREIGN KEY (fk) REFERENCES Parent(pk);`

MÉTHODE Insérer / Modifier / Supprimer des lignes (LMD)

- **INSERT**: `INSERT INTO T (c1, c2) VALUES (v1, v2);` (colonnes optionnelles si toutes fournies dans l'ordre).
- **UPDATE**: `UPDATE T SET c1 = nv1, c2 = nv2 WHERE condition;` **WHERE** obligatoire sauf suppression totale voulue.
- **DELETE**: `DELETE FROM T WHERE condition;` (supprime lignes, garde structure).
`TRUNCATE TABLE T;` (vide table, réinitialise auto-incrément, plus rapide).

MÉTHODE Sélectionner des données (SELECT)

1. **SELECT** **DISTINCT** col1, col2, * FROM Table
2. **WHERE** condition (filtre lignes: =, <>, <, >, <=, >=, BETWEEN, IN, LIKE, IS NULL)
3. **ORDER BY** col1 [ASC|DESC], col2 ... (tri)
4. **LIMIT** n (MySQL) / **TOP** n (Access) – pagination

MÉTHODE Jointure entre deux tables (Équi-jointure)

1. Donner des alias courts aux tables: `FROM Eleve e, Classe c`
2. Préciser la condition de liaison dans **WHERE**: `WHERE e.id_classe = c.id`
3. Préfixer les colonnes ambiguës: `SELECT e.nom, c.libelle`

Syntaxe canonique :

```
SELECT e.nom, e.prenom, c.libelle AS classe
FROM Eleve e, Classe c
WHERE e.id_classe = c.id
      AND c.niveau = 'Terminale'
ORDER BY e.nom;
```

13.5 Opérateurs et prédicats utiles dans WHERE

Catégorie	Opérateur	Exemple / Signification
Comparaison	=, <>, !=, <, >, <=, >=	age >= 18
Logique	AND, OR, NOT	ville='Yaounde' AND age>20
Intervalle	BETWEEN a AND b	note BETWEEN 10 AND 20
Liste	IN (v1, v2, ...)	id_classe IN (1,3,5)
Motif (chaîne)	LIKE '%mot%'	nom LIKE 'M%' (commence par M), _ = 1 caractère
Valeur manquante	IS NULL / IS NOT NULL	telephone IS NULL

À RETENIR Checklist avant le Bac — Leçon 6

- ☐ Je sais définir **BDD**, **SGBD**, **SQL**, **PK**, **FK** et donner un exemple de SGBD (MySQL, Access).
- ☐ Je choisis le bon type : INT (entier), VARCHAR(n) (texte variable), DATE (date), DECIMAL(p,s) (prix/note).
- ☐ J'écris une requête CREATE TABLE avec PRIMARY KEY, NOT NULL, DEFAULT, REFERENCES (FK).
- ☐ J'utilise ALTER TABLE pour ajouter/supprimer une colonne ou une contrainte (FK).
- ☐ J'écris INSERT INTO ... VALUES ... (une ou plusieurs lignes) et INSERT INTO ... SELECT
- ☐ J'écris UPDATE ... SET ... WHERE ... sans oublier le WHERE (sinon mise à jour de toute la table!).
- ☐ J'écris DELETE FROM ... WHERE ... (suppression ciblée) vs TRUNCATE TABLE (vidage total).
- ☐ Je construis un SELECT avec DISTINCT, WHERE (opérateurs + AND/OR/NOT/LIKE/IN/BETWEEN/IS NULL), ORDER BY.
- ☐ Je réalise une jointure 2 tables : alias (e, c), condition WHERE e.fk = c.pk, sélection préfixée.
- ☐ Je distingue **LDD** (structure) et **LMD** (données) et je classe les ordres : CREATE/ALTER/DROP vs INSERT/UPDATE/DELETE/SELECT.
- ☐ Je sais créer une BDD et une table en mode graphique (Access : ruban Créer / Mode Création) et en console MySQL (CREATE DATABASE; USE; CREATE TABLE;).





Signé • OnBuch+

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+